
RESUME DEEP DIVE INTERVIEW MANUAL

Every project. Every claim. Defended.

Diagrams, flowcharts, numbers and scripts

Aditya Moon

Master of Computer Science (Data Science & AI), University of Sydney
Sydney, Australia · adityamoon690@gmail.com · +61 450 800 964

Covers 4 projects, 3 roles, 2 degrees, 1 paper, 1 patent
Purpose Answer anything asked from the CV, in depth
Scope Screening calls through to technical deep dives
Audience Any employer — not tailored to one company

Section 1 tells you how to use this. Section 2 maps every line of the CV to the page that defends it. Sections 3–6 are the four projects in full. Sections 7–10 are experience, research, education and the skills audit. Sections 11–15 are the concepts, the question bank and the cheat sheet.

Read Section 2 first. Print Section 15.

Contents

1	How to use this manual	5
1.1	The five interview modes, and which section serves each	5
1.2	How to answer any question about your own work	5
2	The map: every line of your CV, and where it is defended	7
2.1	Profile and headline claims	7
2.2	Projects	7
2.3	Experience, research and skills	8
3	Your narratives: 30 seconds, 90 seconds, 3 minutes	9
3.1	The through-line	9
3.2	The 30-second version — learn this verbatim	9
3.3	The 90-second version	9
3.4	The 3-minute version — the arc	10
4	Project 1 — GridPulse: Australian energy market data platform	11
4.1	The one-liner, and the three lengths	11
4.2	Why this project exists — the framing that makes it interesting	11
4.3	Architecture — draw this if you get a whiteboard	12
4.4	The ingestion layer in detail	12
4.5	The data model	13
4.6	Orchestration: why Dagster, and what an “asset” is	14
4.7	The 39 tests — know what they actually assert	15
4.8	The story: silent wrongness	16
4.9	The numbers — memorise these	16
4.10	Limitations — volunteer these before you are asked	17
4.11	If they ask “what would you build next?”	17
4.12	Question bank — GridPulse	18
5	Project 2 — Sydney Airbnb Price Intelligence	19
5.1	The one-liner, and the three lengths	19
5.2	Problem framing — the decision most candidates never explain	19
5.3	The dataset	20
5.4	The pipeline	20
5.5	Feature engineering — 24 features, and why each family	21
5.6	Data leakage — the centrepiece story	21

5.7	Results — the full table, not just the headline	23
5.8	The application layer	24
5.9	Engineering around the model	25
5.10	Limitations — volunteer these	25
5.11	Question bank — Airbnb	25
6	Project 3 — SignSpeak: real-time sign language to speech	27
6.1	The one-liner	27
6.2	The recognition pipeline — draw this	27
6.3	Feature engineering — 21 landmarks to 76 dimensions	28
6.4	The pivot: CNN to landmarks	29
6.5	Model selection — accuracy is not the only axis	29
6.6	Numbers and details	30
6.7	Product decisions worth mentioning	30
6.8	Limitations — volunteer these	30
6.9	Question bank — SignSpeak	31
7	Project 4 — Securing large files with hybrid cryptography	32
7.1	The one-liner	32
7.2	Why hybrid — the concept you must have cold	32
7.3	Parallelism — and the thing that makes it non-trivial	33
7.4	Question bank — cryptography	33
8	Experience — three roles, and what each one proves	35
8.1	Magic Software Enterprises — Software Developer (internship)	35
8.2	Hewlett Packard Enterprise — Technical Consultant	36
8.3	Virtuous Soft Solutions — IT Instructor	36
9	Research and patent	38
9.1	IEEE ICBDS 2023 — Security, Privacy and Scalability in Blockchain and IoT	38
9.2	Granted patent — IoT simulation and cloud integration system	38
9.3	Top Project Award — Project Management in IT, University of Sydney	39
10	Education	40
10.1	Master of Computer Science (Data Science and AI) — University of Sydney	40
10.2	Bachelor of Computer Engineering — VIIT Pune	40
10.3	Availability and work rights — say this cleanly, once	40
11	The skills audit — every claim on the CV, graded	41
11.1	AI and LLMs	41
11.2	ML engineering	41

11.3 Data engineering	42
11.4 Production and cloud — the one to be most careful with	42
11.5 Product, UI/UX, security	42
12 Concept refreshers — the fundamentals under your CV	43
12.1 The nesting: AI, ML, deep learning, LLMs	43
12.2 Supervised, unsupervised, reinforcement	43
12.3 Overfitting, underfitting, and the bias–variance trade-off	43
12.4 The confusion matrix and the metrics that come from it	44
12.5 The model families, one line each	45
12.6 LLMs — enough to be credible without overclaiming	45
12.7 Data engineering vocabulary	47
12.8 SQL — expect to write some	47
12.9 Software engineering vocabulary	48
12.10 Security and privacy vocabulary	48
12.11 The system-design answer — a reusable template	48
13 Behavioural questions — your stories, mapped	50
13.1 STAR, and the ratio that matters	50
13.2 The story matrix — one table to cover the whole behavioural round	50
14 The general question bank	52
14.1 Motivation and fit	52
14.2 Technical judgement	52
14.3 Curveballs	53
15 Questions you ask them	54
15.1 The strongest three	54
15.2 The reserve list	54
15.3 Save these for later stages	54
16 The consistency audit — do this before any interview	55
16.1 A four-day preparation plan	55
17 The one page — print this	56

1. How to use this manual

This is not a document to read once. It is a reference you drill from, in three passes.

1. **Pass one — the map.** Read Section 2. It lists every claim on your CV and points at the page that defends it. If a line on your CV has no defence, that is the line you will be asked about.
2. **Pass two — the projects.** Sections 4–7. For each project: read the diagram, then say the 60-second version out loud without looking. Then read the question bank and answer each one out loud before reading the answer.
3. **Pass three — the night before.** Section 17. One page. Numbers, one-liners, and the three sentences you must not fumble.

The single rule that governs everything here

You will be asked about the thing you wrote down, at the depth you wrote it.

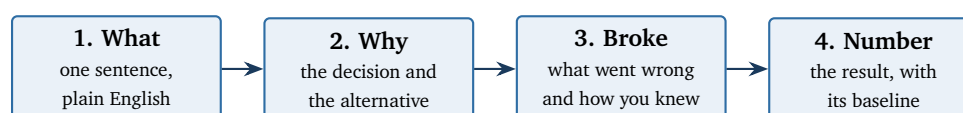
If your CV says “39 automated data-quality tests”, a competent interviewer asks “what does one of them actually assert?”. If it says “0.93 ROC-AUC”, they ask “what is ROC-AUC and why did you report it instead of accuracy?”. Every number on a CV is a promise that you can go one level deeper than the number. This manual is that level.

1.1 The five interview modes, and which section serves each

Mode	What they are really testing	Read
Recruiter screen (15–20 min)	Can you describe what you did in plain English, and do the dates and the visa work	§3, §17
Hiring-manager chat (30–45 min)	Judgement. Why did you make the choices you made, and what went wrong	Project “design decisions” and “what broke” subsections
Technical deep dive (45–60 min)	Depth on one thing you claim to own; can you draw the architecture	§4–§7, all diagrams
Fundamentals / theory	Do you understand the concepts underneath the tools	§12
Behavioural / values	Ownership, conflict, failure, communication	§8, §13

1.2 How to answer any question about your own work

Use this four-beat structure. It is short enough to hold under pressure and it lands the things interviewers actually score.



Beats 2 and 3 are where the marks are. Beat 1 is where most candidates spend all their time.

The three answers that lose you the job

- **“We did...”** — an interviewer cannot score a “we”. Say “I”. If it was genuinely shared, say “the team did X; my part was Y”.
- **Bluffing a term.** If you do not know what a hyperparameter is called, say so and ask. Inventing it is the only unrecoverable move, and experienced interviewers always notice.
- **A number with no baseline.** “80% accuracy” sounds mediocre. “80% against a 39% majority-class

baseline” is a real result. Never say the first without the second.

2. The map: every line of your CV, and where it is defended

Read down the left column. If you cannot immediately produce the middle column from memory, go to the section in the right column.

2.1 Profile and headline claims

CV says	You must be able to say	Section
“Master of Computer Science (Data Science & AI), Dec 2026”	Which units, which one produced which project, and what your thesis/major work is	§10
“three ML and data products live in production”	Name them, give each a URL, and say what “in production” means for each (public URL, CI, tests, reproducible build)	§4–§6
“automated tests, CI/CD”	What runs on a push, what fails a build, and one test you wrote that caught a real bug	§12
“rule-driven validation, auditable data lineage”	The GridPulse test taxonomy and the Dagster asset graph	§4
“honest model evaluation”	The leakage story, and why the score <i>went down</i>	§5

2.2 Projects

CV says	You must be able to say	Section
GridPulse: “~668K rows, 541 facilities”	Where the numbers come from (OpenElectricity v4, 5-minute grain, 7-day window) and what one row is	§4.9
“retry/backoff, rate-limit handling, offline caching”	Exponential backoff, why you cache raw JSON before parsing, and how that makes the build reproducible without an API key	§4.4
“39 automated data-quality tests”	The five families of test, and three concrete examples	§4.7
“star-schema marts”	Facts vs dimensions, your grain, and why you modelled it that way	§4.5
“orchestrated as Dagster assets”	Asset vs task orchestration, and why it gives you lineage for free	§4.6
Airbnb: “17.7K listings”	Inside Airbnb, June 2026 snapshot, asking prices not transactions	§5.3
“leakage-safe preprocessing”	What leaked, how you noticed, the Pipeline fix, and the test that prevents recurrence	§5.6
“80% test accuracy (0.93 ROC-AUC) against a 39% baseline”	The full model table, macro-F1, what ROC-AUC means, and where the errors sit	§5.7
“confidence-scored, explainable recommendations”	How the confidence number is produced and what it does <i>not</i> mean	§5.8
SignSpeak: “MediaPipe + SVM, 24 ASL letters”	Why 24 not 26, the 21 landmarks, the 76-dim feature vector	§6.3
“re-architected a heavyweight CNN”	Why the CNN was fragile, and why better features beat a bigger model	§6.4
“99.2% test accuracy”	Check this figure against the repo before you quote it — see the consistency audit	§16
Crypto: “AES+RSA, multi-threaded, ~3× faster”	Why hybrid at all, what RSA actually encrypts, chunking and thread safety	§7

2.3 Experience, research and skills

CV says	You must be able to say	Section
HPE: “20+ customers, 90%+ within SLA, ~30% faster resolution”	What an SLA is, one hard ticket end to end, and how you measured the 30%	§8.2
Magic: “5+ C++ features, ~40% throughput, 30+ defects”	One feature in design detail, and how you debug code you did not write	§8.1
Virtuous Soft: “50+ learners, 15+ projects”	How you explain a hard idea to a non-technical person — expect to be asked to demonstrate it live	§8.3
IEEE ICBDS 2023, cited by 7	Your contribution specifically, the paper’s argument, and one limitation of it	§9
Granted patent (South Africa, 2023)	What the claim covers, and what a granted patent does and does not mean	§9.2
“AWS core services — foundational”	The word “foundational” is doing work here. Know what EC2/S3/Lambda/RDS/IAM are for; do not claim production AWS	§11
“LLM application development, RAG, prompt engineering”	Be precise about what you have built vs studied	§11, §12.6

3. Your narratives: 30 seconds, 90 seconds, 3 minutes

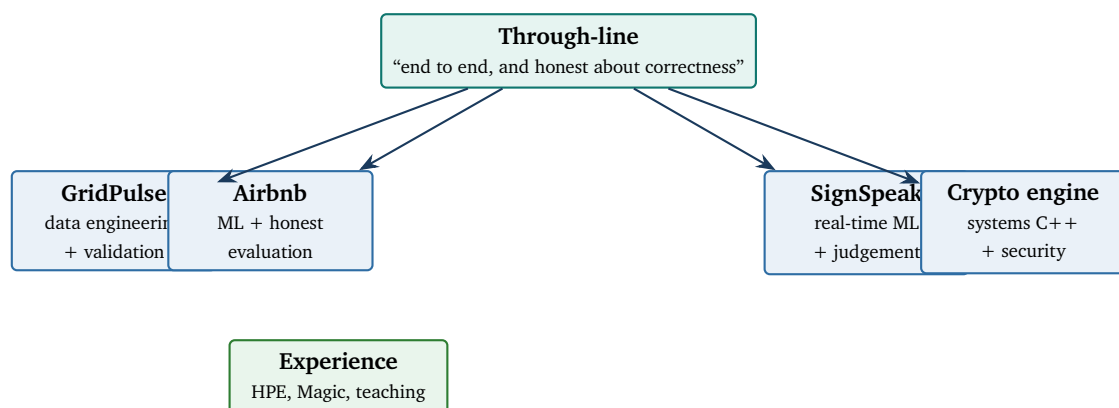
You need three lengths of the same story, because you do not control how much time you get. Learn the 30-second version verbatim. Learn the shape of the other two.

3.1 The through-line

Every good candidate has one sentence that organises everything else. Yours is:

“I build AI systems end to end — and I care most about the part that makes them trustworthy: knowing whether the number is actually right.”

That sentence is earned, not asserted, because you have two stories where the honest answer was uncomfortable: a pipeline that published wrong numbers silently, and a model whose score you had to reduce.



Every answer you give should be traceable back to the box at the top.

3.2 The 30-second version — learn this verbatim

30 seconds — “tell me about yourself”

“I’m finishing a Master of Computer Science in Data Science and AI at the University of Sydney in December 2026. Before that I worked as a C++ developer at Magic Software and as a technical consultant at HPE, in India.

What I actually do is build AI and data systems end to end and put them in front of people — I’ve got three live applications: a data platform over the Australian electricity market, a machine-learning pricing tool for Sydney Airbnb hosts, and a real-time sign-language-to-speech app.

The thread through all of them is that I care about whether the number is right, not just whether the demo works. Both of the interesting stories I have are about catching my own system being confidently wrong.”

3.3 The 90-second version

Same opening, then add one project in depth — pick by audience:

If the role is...	Lead with
Data engineering / analytics	GridPulse. The silent-partial-load story.
ML / data science	Airbnb. The leakage story.
AI / LLM product	Airbnb (evaluation discipline) + your interest in retrieval and structured output. Be honest about what you have shipped.
Backend / systems / C++	The cryptography engine + Magic Software.
Anything at a small company or startup	GridPulse + “I’ve taught, I’ve supported customers, I’ve shipped C++ — I can move between the pieces of a small team’s stack.”

3.4 The 3-minute version — the arc

1. **Where I started** (20s). Computer Engineering at VIIT Pune, 9.4/10. Wrote a published paper and a granted patent as an undergraduate — I liked the research end early.
2. **What industry taught me** (40s). Magic Software: shipping C++ into a commercial product, and thirty-plus bug reports that arrived as “it’s slow on Tuesdays” rather than stack traces. HPE: twenty-plus enterprise customers, where the skill is understanding a frustrated person before solving their problem.
3. **Why I moved to AI** (30s). I wanted to build the systems, not support them. Masters at Sydney in Data Science and AI.
4. **What I’ve built since** (60s). The three live products — one sentence each, then one in depth.
5. **What I want next** (20s). Ownership. Shipping to production early rather than rotating through a graduate program.

Trap

Do not narrate your CV chronologically for three minutes. Interviewers stop listening at about 90 seconds. The arc above works because every beat answers “so what?”. If you find yourself saying “and then I did...” more than twice, you have lost them.

The transition question you will definitely be asked

“You were doing enterprise support and C++ — why AI?”

Have this ready and do not sound defensive:

“At Magic I was writing the encryption feature and at HPE I was supporting systems other people had built. Both were interesting, but I kept wanting to be further up the stack — deciding what the system should do, not just making it run. The masters was a deliberate move for that, and the three projects since are me proving I can do the whole thing, not just the modelling.”

4. Project 1 — GridPulse: Australian energy market data platform

Repo: github.com/adhu-601/gridpulse **Live:** gridpulse-nem-analytics.streamlit.app

Stack: Python 3.12, DuckDB, dbt (dbt-duckdb), Dagster, Streamlit + Plotly, pandas, pytest

Origin: grew out of COMP5339 Data Engineering at the University of Sydney, then rebuilt as a packaged, orchestrated, tested system. Co-authored with Pranjal Desai — *be explicit about which parts were yours.*

4.1 The one-liner, and the three lengths

15 seconds

“GridPulse is an end-to-end data platform over the Australian National Electricity Market. It pulls five-minute generation and emissions data for every registered facility, models it in a DuckDB warehouse with dbt, and serves it as a live dashboard — with the whole lineage orchestrated in Dagster and 39 data-quality tests gating publication.”

60 seconds

“It ingests five-minute generation and emissions readings for 541 registered facilities across the five NEM regions — about 668,000 records over a seven-day window — from the OpenElectricity v4 API. The API is unreliable in ordinary ways: rate limits, timeouts, partial responses. So a lot of the work is retries with exponential backoff, batching, and writing the raw JSON to disk *before* parsing anything, so the whole warehouse can be rebuilt offline without an API key.

From there it's nine Python-level quality gates before load, a DuckDB warehouse, dbt marts on a star schema, and 39 dbt tests over those marts. Dagster runs the whole thing as one asset graph, which means every figure on the dashboard traces back to the source that produced it.

The headline finding is a nice one: coal supplies around 58% of the energy in that window and roughly 95% of the emissions.”

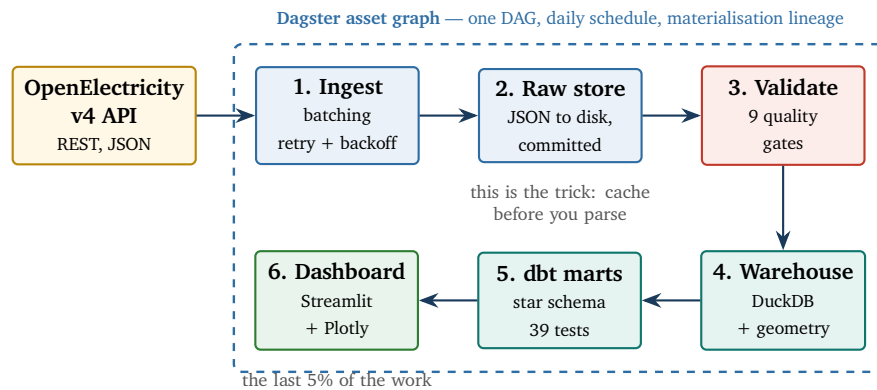
4.2 Why this project exists — the framing that makes it interesting

Do not describe it as “a dashboard”. Describe it as **a system for publishing numbers you are willing to defend**. The dashboard is the last five percent.

The problem statement, in interview language

“Anyone can pull an API into a chart. The hard part is that the API will fail halfway through one day, and a half-finished load does not raise an error — it just aggregates fewer rows and publishes a completely plausible wrong number. **A crash is obvious; a quietly wrong number gets believed.** GridPulse is built around making that failure impossible to publish.”

4.3 Architecture — draw this if you get a whiteboard



4.3.1 Layer by layer, and why each one exists

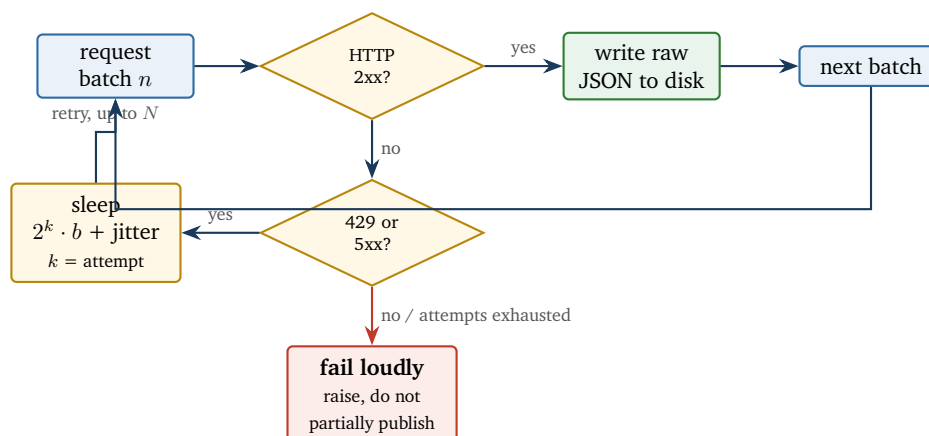
Layer	What it does, and the reason it is separate
Ingest	Calls the API in batches with retry, exponential backoff and rate-limit handling. Separate because network failure is a different failure class from data failure and should be retried, not diagnosed.
Raw store	Writes the untouched JSON response to disk and commits it. Separate because it makes the build <i>reproducible</i> — anyone can rebuild every downstream artifact with no API key — and because when a number looks wrong you can go back to exactly the bytes the API returned.
Validate	Nine Python-level checks run <i>before</i> anything lands in the warehouse: row count, schema, nulls, grain, domain, timestamps, emissions sanity, coordinate sanity. Separate because the warehouse should never contain data you have not agreed to.
Warehouse	DuckDB, with geometry support for facility coordinates. This is the single source of truth.
dbt marts	SQL transformations into a star schema, plus 39 declarative tests. Separate because business logic in SQL is reviewable and version-controlled; business logic buried in a dashboard is not.
Dashboard	Streamlit + Plotly. Reads marts only. It contains no business logic — deliberately.

The principle behind the layering: ELT, not ETL

ETL transforms data *before* loading it. **ELT** loads raw data first, then transforms inside the warehouse. GridPulse is ELT, and the reason matters: if your transform logic is wrong, ETL means you have lost the original and must re-pull from an API that may have changed. ELT means you re-run SQL against data you already hold. **Raw data is cheap to keep and expensive to re-acquire.**

4.4 The ingestion layer in detail

This is the part most interviewers probe, because every company has an integration that misbehaves.



Term	Definition you should be able to give
Retry	Re-issuing a failed request. Only safe for idempotent operations — a GET is idempotent, a POST that charges a card is not. All of GridPulse's calls are reads, so retry is free.
Exponential backoff	Wait 2^k times a base delay after the k th failure, rather than a fixed delay. A fixed delay from many clients synchronises into a thundering herd; doubling spreads the load out.
Jitter	Adding a small random amount to each backoff so that clients that failed together do not retry together.
Rate limit (HTTP 429)	The server telling you to slow down. Distinct from an error: it is not a failure, it is a request to wait — often with a Retry-After header you should obey rather than guess.
Idempotency	Doing the operation twice has the same effect as doing it once. The property that makes retries safe.
Batching	Requesting many records per call rather than one at a time — fewer round trips, fewer chances to be rate-limited.

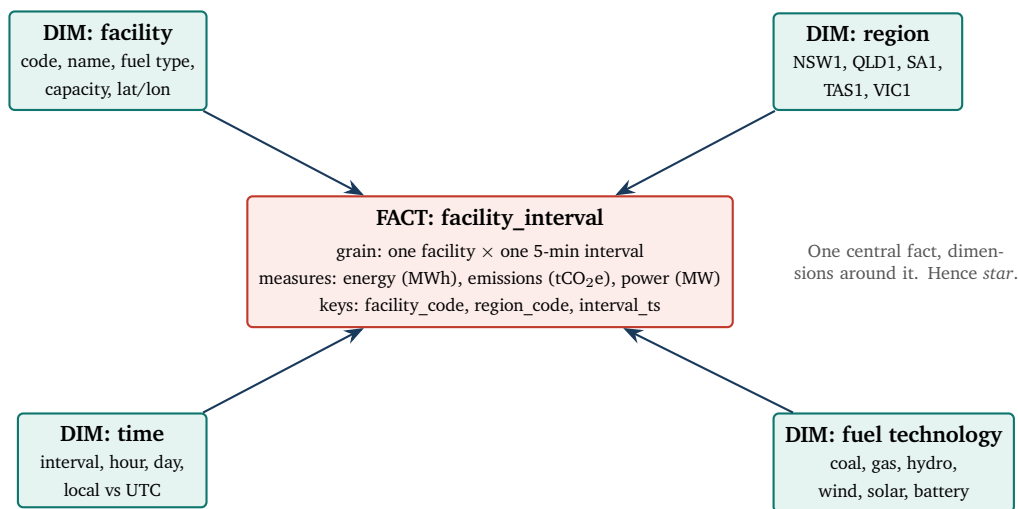
Why the raw cache is the smartest decision in the project

Committing the raw JSON to the repository buys you four things at once:

1. **Reproducibility.** `python -m gridpulse.pipeline` rebuilds every artifact with no API key and no network.
2. **Auditability.** When a figure is questioned, you can point at the exact bytes the source returned.
3. **Fast iteration.** You can rewrite the transform twenty times without hitting the API once.
4. **Determinism in tests.** CI runs against a fixed snapshot, so a test failure is always your code, never the weather.

The cost is repository size and staleness — which is exactly the trade-off to name if asked.

4.5 The data model

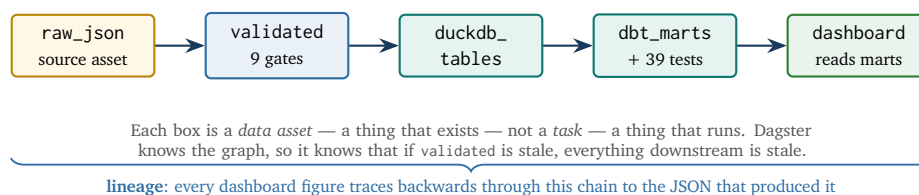


Term	Definition
Fact table	The measurements. Long and narrow, one row per event. Numeric, additive columns.
Dimension table	The descriptive context you slice by. Short and wide, one row per entity.
Grain	<i>The single most important sentence in any data model:</i> what exactly does one row represent? Here: one facility, one five-minute interval. Get the grain wrong and every aggregate is wrong.
Star schema	Fact in the middle, dimensions radiating out, joined by keys. Optimised for analytical queries and for humans reading the SQL.
Snowflake schema	A star where dimensions are themselves normalised into sub-tables. Less duplication, more joins, harder to read. Usually not worth it for analytics.
Normalisation	Removing duplication so each fact is stored once. Right for transactional systems (OLTP), often wrong for analytics (OLAP) where join cost dominates.
Additive measure	One you can sum across any dimension — energy in MWh. <i>Non-additive</i> : a rate like carbon intensity, which you must recompute from its numerator and denominator rather than average.

The trap question: “can you average carbon intensity across regions?”

No — and this is a genuinely good question to be asked. Carbon intensity is tCO₂e per MWh, a ratio. The average of ratios is not the ratio of sums. You must sum the emissions, sum the energy, then divide. Averaging region-level intensities weights a tiny region equally with a huge one. If you can say this cleanly you have demonstrated you understand your own data model rather than having copied a tutorial.

4.6 Orchestration: why Dagster, and what an “asset” is



Term	Definition
Orchestration	Deciding what runs, in what order, on what schedule, and what happens when a step fails.
DAG	Directed Acyclic Graph — steps with dependencies and no cycles. The shape every pipeline has.
Task orchestration	The Airflow default: you define <i>operations</i> . “Run this script, then that one.”
Asset orchestration	The Dagster model: you define <i>the data that should exist</i> and how to produce it. The scheduler works out the operations.
Materialisation	The act of (re)computing an asset. Dagster records when each asset was last materialised and from which upstream versions.
Lineage / prove-nance	The traceable chain from a published figure back to its source. This is the word to use when a company cares about audit or regulation.
Backfill	Re-running a pipeline over historical periods, usually after fixing a bug. Much easier when assets are partitioned by date.

“Why Dagster and not Airflow? Or just cron?”

Cron was the honest starting point and it fails for one reason: it has no concept of dependencies or of failure propagating. If step two fails, cron happily runs step three.

Airflow is the industry default and would have worked. It is task-centric, which means lineage is something you document rather than something the tool knows.

Dagster models assets rather than tasks, so the lineage graph is the program. Since the whole

point of the project was “every published figure is traceable and validated”, having the orchestrator natively understand data dependencies — and having dbt models appear as assets in the same graph — was the right fit. It is also far lighter to run locally, which matters for a project that has to be reproducible by anyone who clones it.

Then add the honest note: “At production scale with a big team, I’d expect the choice to come down to what the team already runs. I wouldn’t argue Dagster over Airflow on principle.”

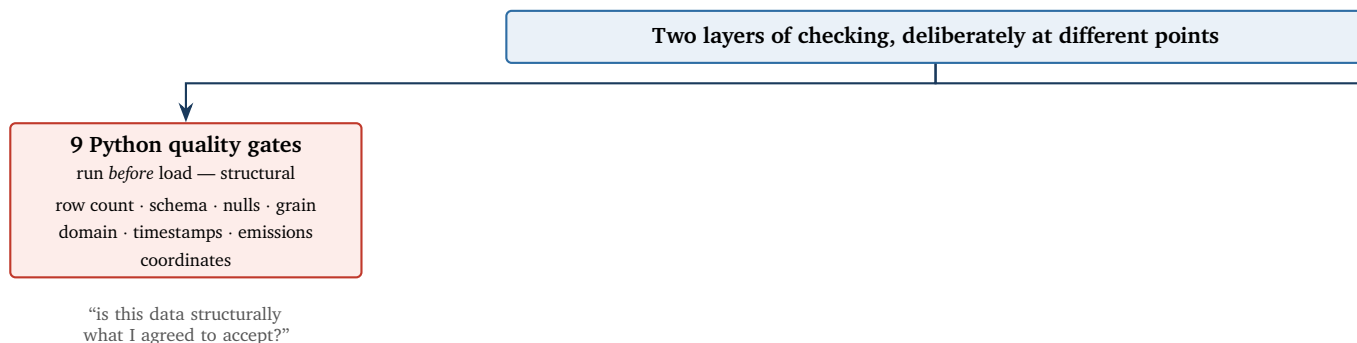
“Why DuckDB and not Postgres, Snowflake or BigQuery?”

DuckDB is an **embedded OLAP** database — think “SQLite for analytics”. It runs in-process, stores columnar data in a single file, and is extremely fast for scans and aggregations over hundreds of millions of rows on a laptop.

- **vs Postgres:** Postgres is row-oriented and built for transactions. Aggregating 668K+ rows across columns is exactly what a columnar engine is for.
- **vs Snowflake/BigQuery:** they are the right answer at terabyte scale with many concurrent users. They also cost money, need credentials, and would have made the project impossible to reproduce by cloning a repo.
- **The limit, stated up front:** DuckDB is single-writer and file-based. It is not a multi-user serving database. If GridPulse needed concurrent writers or a shared team warehouse, this decision would have to change — and that is a scale problem, not a correctness problem.

4.7 The 39 tests — know what they actually assert

Do not say “I wrote 39 tests” and stop. Say what kind, and give examples.



Family	dbt name	Concrete example from GridPulse
Uniqueness	unique	facility_code appears once in the facility dimension. Duplicates would double-count energy.
Presence	not_null	Every interval row has a non-null timestamp and region.
Referential integrity	relationships	Every facility_code in the fact table exists in the facility dimension — catches an orphaned reading from a facility the dimension load missed.
Categorical domain	accepted_values	region_code is one of NSW1, QLD1, SA1, TAS1, VIC1. A sixth value means the API changed under you.
Numeric domain	accepted_range	Emissions ≥ 0 ; capacity within plausible bounds; latitude/longitude inside Australia.
Custom / singular	bespoke SQL	Expected row count for a complete window; no gaps in the five-minute series; energy reconciles against power $\times$ interval length.

The distinction that impresses people: unit tests vs data tests

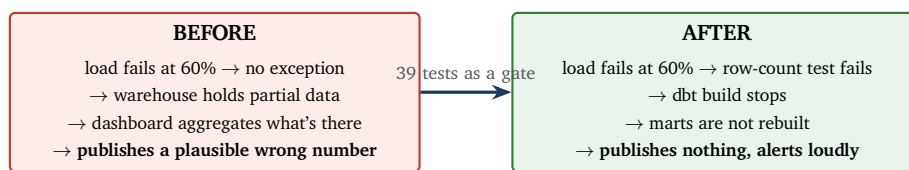
A unit test tests your code with data you invented. It answers: “given this input, does my function behave?” It is deterministic and it passes forever once written.

A data test tests real data against rules you asserted. It answers: “is what actually arrived today still what I assumed?” It can pass for a year and then fail because the *world* changed — a new region code, a schema change, a facility retired.

You need both, and they fail for completely different reasons. GridPulse has pytest for the first and dbt tests for the second. Most people conflate them; saying this clearly signals real pipeline experience.

4.8 The story: silent wrongness

This is your best engineering story. Learn its shape, not its words.



90 seconds — the GridPulse story

“The build itself was mostly about the API misbehaving in ordinary ways — rate limits, timeouts, partial responses — so a lot of the work went into retries, backoff and caching so it survives a bad afternoon unattended.

But the thing I actually learned came later. I noticed some figures looked slightly off, and when I traced it back, a scheduled load had failed partway through. It hadn’t thrown an error. It had just written fewer rows, and the dashboard cheerfully aggregated what was there and published a number that looked completely plausible and was wrong.

That’s the failure mode that scares me. A crash is obvious and someone fixes it. A quietly wrong number gets believed. So I built the data-quality tests around the pipeline — expected row counts, date coverage, no impossible values, referential integrity — so a partial load fails loudly and publishes nothing at all rather than publishing something half-right.”

Why this story travels to any company

Every business has a version of this. A billing pipeline that undercounts. A reporting job that drops a region. A recommender trained on half a day’s events. The lesson generalises: **the dangerous failures are the silent ones, and the fix is to make correctness a gate rather than an observation.** Land the phrase: “a crash is obvious; a quietly wrong number gets believed.”

4.9 The numbers — memorise these

GridPulse facts

Readings	668,134
Facilities	541 registered generators
Regions	5 (NSW1, QLD1, SA1, TAS1, VIC1)
Grain	5-minute intervals
Window	7 days, 12–18 May 2026
Energy generated	3.62 TWh
Emissions	2.15 Mt CO ₂ e
Fossil share	~64% of energy, ~100% of emissions
Coal specifically	~58% of energy, ~95% of emissions
Carbon intensity range	VIC1 0.767 → TAS1 0.013 tCO ₂ e/MWh
Quality gates	9 of 9 passing (Python, pre-load)
dbt data tests	39 of 39 passing
Source	OpenElectricity v4 API, CC BY 4.0

Have an opinion about the finding, not just the number

The best follow-up you can give: “VIC1 at 0.767 and TAS1 at 0.013 is a nearly sixty-fold difference in carbon intensity between two states in the same market — Victoria’s brown coal against Tasmania’s hydro. It’s a good illustration of why a national average is almost useless for policy: the number that matters is regional and it moves by the hour.”

Interviewers remember candidates who understood their data. Anyone can recite a metric.

4.10 Limitations — volunteer these before you are asked

- **It is a seven-day snapshot**, not a long-run average. Seasonality, outages and price events are invisible at that window.
- **Operational metering, not settlement data.** These are aggregated five-minute readings, not the audited figures AEMO settles on. Close, but not the same thing, and that distinction matters if anyone were to make a financial decision on it.
- **Full refresh, not incremental.** dbt models rebuild rather than appending. Fine at 668K rows; wrong at 668M. The fix is incremental models with partitioned Parquet, which is the top item on the roadmap.
- **Single-writer warehouse.** DuckDB is not a shared serving database.
- **CI does not yet run the full pipeline.** Running pytest and dbt test on every commit is the second roadmap item.

Why volunteering limitations wins

Naming the boundary of your own work is the clearest available signal of seniority. It says: I know what this is for and what it is not for. Candidates who oversell get found out in the follow-up question; candidates who self-limit get trusted with the next question.

4.11 If they ask “what would you build next?”

1. **Incremental models and partitioning.** Move from full refresh to incremental dbt models over date-partitioned Parquet, so multi-week and multi-year history becomes tractable. This is the single change that makes it a real platform rather than a snapshot.
2. **CI running the whole thing.** pytest plus dbt build on every push, against the committed raw cache, so the tests that gate publication also gate merging.
3. **Freshness and volume alerting.** dbt’s source freshness plus an anomaly check on row counts — “today’s load is 40% smaller than the trailing median” is the cheapest early warning there is.
4. **Short-horizon forecasting.** Price and emissions-intensity forecasting on top of the marts — but only

after the history exists, because you cannot forecast from a week.

4.12 Question bank — GridPulse

Q. What is one row in your fact table?

A. One facility's reading for one five-minute interval, carrying energy in MWh, power in MW and emissions in tCO₂e, keyed to facility, region and timestamp. That sentence is the grain, and everything downstream depends on it being unambiguous.

Q. How do you know the pipeline is correct?

A. I don't rely on knowing — I rely on it being checked. Nine structural gates before load and 39 declarative tests after transform. The important design choice is that they are a *gate*: if they fail, the marts are not rebuilt and the dashboard keeps serving the last known-good state rather than a fresh wrong one.

Q. The API returns a 429 halfway through. What happens?

A. The client backs off exponentially with jitter and retries up to a bounded number of attempts. If it still fails, the run raises rather than continuing — because the alternative is a partial load, which is the exact failure this project is built around. And because everything is a read, retrying is safe: the calls are idempotent.

Q. What if the API adds a new region code tomorrow?

A. The `accepted_values` test on `region_code` fails and the build stops. That is the intended behaviour — it is not a bug that a schema change breaks my pipeline, it is a feature that it breaks it *loudly* instead of silently misclassifying rows.

Q. Why not put the business logic in the dashboard? It would be simpler.

A. It would be simpler once and worse forever. Logic in SQL models is version-controlled, reviewable, testable and reusable by anything downstream. Logic in a Streamlit callback is none of those, and it means the definition of “carbon intensity” lives in a place nobody can audit.

Q. How would you scale this to five years of data instead of seven days?

A. Three changes. Partition the raw layer by date and store it as Parquet rather than JSON. Convert the dbt models from full refresh to incremental with a merge on the interval key. Then reconsider the warehouse — DuckDB will still handle it on a decent machine, but if multiple people need concurrent access it becomes the constraint, and I'd move to something like BigQuery or Postgres with a columnar extension.

Q. You co-authored this. What was yours?

A. *Answer this precisely and honestly — name your specific components.* A vague answer here is worse than a modest one. Say which layers you owned, which you pair-worked, and which your collaborator led.

Q. What is the hardest bug you hit on it?

A. The silent partial load — see §4.8. Hardest because there was no error to follow; the only signal was a figure that looked slightly wrong, and the investigation was backwards from the number rather than forwards from a stack trace.

5. Project 2 — Sydney Airbnb Price Intelligence

Repo: github.com/adhu-601/sydney-airbnb-price-intelligence **Live:** sydney-airbnb-price-intelligence.streamlit.app

Stack: Python, scikit-learn, XGBoost, pandas, Streamlit, pytest, GitHub Actions

Origin: a University of Sydney research study, rebuilt as an installable package with a CLI, a serving class, a test suite and CI.

5.1 The one-liner, and the three lengths

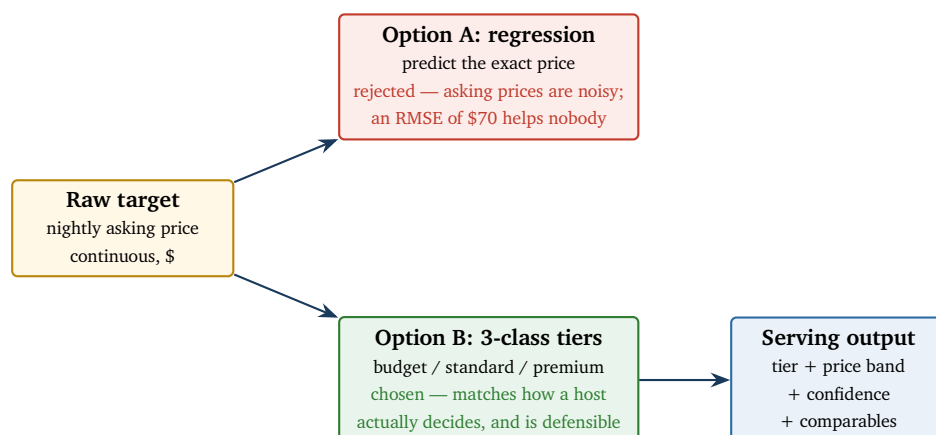
15 seconds

“It’s a pricing decision tool for Sydney Airbnb hosts. Seventeen and a half thousand listings, an XGBoost classifier that places a listing in a market tier, and a live app that turns that into a confidence-scored price band with the comparable listings behind it.”

60 seconds

“I took a university research study to production. The data is the Inside Airbnb Sydney snapshot — 17,701 live listings. Rather than predicting an exact nightly price, which is a low-signal regression problem on asking prices, I framed it as a three-class tier prediction: budget under \$200, standard \$200–400, premium above \$400, thresholds set at market terciles so the classes are roughly balanced. Twenty-four features, mostly geospatial and amenity-based — Haversine distance to the CBD and to five major beaches, amenity counts, host scale, review signals. XGBoost came out best at 79.8% test accuracy and 0.930 macro ROC-AUC against a 39% majority-class baseline. The part I’d actually want to talk about is that the first version of that number was a lie — I had data leakage, and fixing it made the score go down.”

5.2 Problem framing — the decision most candidates never explain



Why tiers, and why terciles — have this answer ready

- **Why not regression?** The target is an *asking* price, not a transacted one. Two identical apartments can list \$100 apart because one host is optimistic. Predicting that noise to the dollar is fitting the host’s mood, not the market. A tier is the level at which the signal is real.
- **Why three classes?** It is the smallest number that gives a host an actionable decision (“you’re pricing as budget, the market says standard”) without pretending to a precision the data does not

support.

- **Why terciles?** Setting cut-points at roughly the 33rd and 67th percentiles gives 31/39/30 class balance. That matters twice: an imbalanced problem needs different metrics and resampling, and *round-number* thresholds (\$200, \$400) are interpretable to a host in a way that “\$213.40” is not. I got both by choosing thresholds that were near the terciles and also human-readable.
- **The honest limitation:** the thresholds are a modelling choice, not a fact about the world. A listing at \$199 and one at \$201 are labelled differently and are basically the same. That is why the model reports a confidence and why almost all its errors are between adjacent tiers.

5.3 The dataset

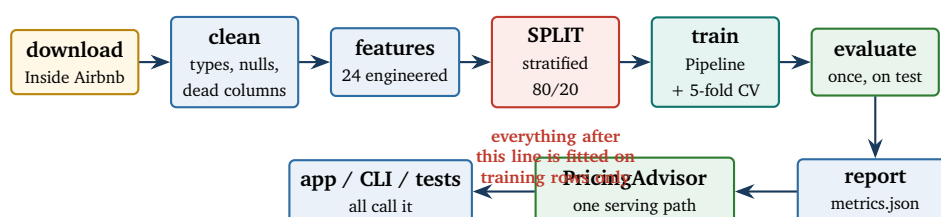
Airbnb data facts

Source	Inside Airbnb, Sydney snapshot, 16 June 2026, CC BY 4.0
Listings	17,701 live listings
Target	nightly asking price → 3 tiers
Tiers	Budget <\$200 · Standard \$200–400 · Premium >\$400
Class balance	31% / 39% / 30%
Features	24 engineered
Split	stratified 80/20, then 5-fold CV inside the training split
Test set size	3,541 listings
Retrieval	reproducible via <code>airbnb-pricer</code> download

Know what your data is not

Inside Airbnb scrapes *listed* prices. It does not observe bookings, occupancy or revenue. So the model learns “what does a listing like this ask?”, not “what should this listing charge to maximise revenue?”. Those are different questions and conflating them would be the substantive criticism of the project. Say it before they do — and note that the app frames its output as decision support, not financial advice, for exactly that reason.

5.4 The pipeline



The single most important design decision: one serving path

PricingAdvisor is the only class that turns a listing into a prediction, and the CLI, the Streamlit app and the tests all call it. That means **training/serving skew** — where the app computes a feature slightly differently from the training code and the model silently degrades — is structurally impossible rather than merely unlikely.

This is a genuinely production-minded decision and it is worth volunteering. Training/serving skew is one of the most common ways real ML systems fail quietly, and it is very hard to detect from metrics because your offline evaluation never sees it.

5.5 Feature engineering — 24 features, and why each family

Family	Examples	Why it should carry signal
Geospatial	Haversine distance to CBD; distance to five major beaches	Location is the dominant driver of accommodation price everywhere. A raw lat/lon pair is nearly useless to a tree; a <i>distance</i> is directly interpretable and monotonic.
Amenities	total count + six binary flags (pool, air-con, parking, ...)	Count captures generosity of the listing; the individual flags capture the handful that actually command a premium.
Host	host scale (number of listings), superhost status	Distinguishes a professional operator from someone letting a spare room — they price differently.
Reviews	volume and quality metrics	Proxy for tenure and demand. Beware: partly an <i>outcome</i> of price, which is worth acknowledging.
Capacity	bucketed bedrooms, guests accommodated	Bucketing reduces the influence of rare extreme values (a 14-bedroom listing) without discarding them.
Property type	rare types collapsed into “Other”	Prevents dozens of one-row categories from creating noise dimensions after encoding.

Haversine distance — be able to define it

The great-circle distance between two lat/lon points on a sphere. Ordinary Euclidean distance on raw latitude and longitude is wrong because a degree of longitude is not a fixed number of kilometres — it shrinks as you move away from the equator. Haversine gives real kilometres. Using it rather than a naive coordinate difference is a small detail that shows you thought about the geometry.

“Aren’t review counts leakage?”

This is the sharpest question you can be asked about the feature set, and the honest answer is better than a defensive one:

“Not leakage in the strict sense — review count is observable at prediction time, so it doesn’t violate temporal availability. But it is partly an effect of price rather than a cause: cheaper listings book more and accumulate reviews faster. So it’s a legitimate predictive feature and a poor causal one. The model is a market-comparison tool, not a causal estimate of what happens if you raise your price — and I’d be very careful about anyone reading it that way.”

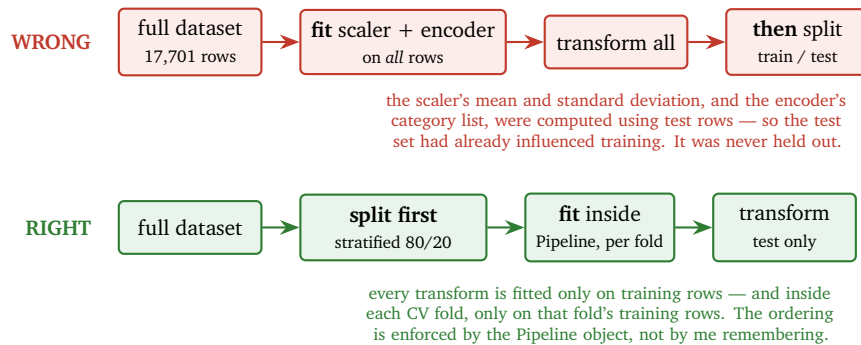
5.6 Data leakage — the centrepiece story

5.6.1 What leakage is

Data leakage

Information that would not be available at prediction time influencing the model during training. The symptom is an evaluation score that is better than the model’s real ability. The danger is that it is a *flattering* bug: nothing crashes, nothing looks wrong, and every incentive you have is to believe it.

5.6.2 Exactly what happened here



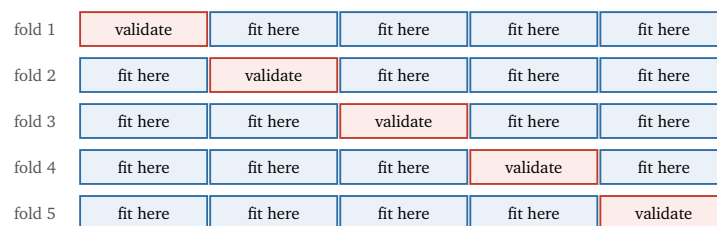
5.6.3 Why a Pipeline object is the real fix

Fixing a bug vs making it unrepeatable

Splitting before fitting is the *fix*. Wrapping preprocessing and model in a `sklearn.pipeline.Pipeline` is the *prevention*, because it changes the failure from “I must remember the right order” to “the wrong order is not expressible”.

When you call `cross_val_score` on a Pipeline, scikit-learn re-fits every transformer inside each fold, on that fold's training rows only. You cannot leak across folds without deliberately dismantling it.

This is the general principle worth stating out loud: the best fix for a class of mistake is a structure that makes the mistake impossible, not a note to yourself to be careful. That applies far beyond ML.



5-fold cross-validation. The whole preprocessing pipeline is re-fitted on the blue blocks in every row. Averaging five scores also tells you how *stable* the model is — a large spread across folds is itself a finding.

5.6.4 Telling the story

90 seconds — the leakage story

“I turned a university research study into a working pricing tool for Sydney Airbnb listings — about 17,700 listings, predicting a price band, deployed as a live app that gives hosts a confidence-scored recommendation.

The first results looked great, and that was the problem. When something jumps that far above your baseline you should be suspicious rather than pleased, and I wasn't at first.

What had happened was that I'd done my preprocessing — the scaling and the category encoding — before splitting the data into training and test sets. So the transformation had been fitted using the test rows as well. The test set wasn't genuinely unseen any more, and the score was flattering rather than honest.

The fix itself was small: split first, fit the preprocessing on the training data only, then apply it to the test set — and put the whole thing inside a Pipeline object so that ordering is enforced by the structure rather than by me remembering. The real fix was the automated tests I added, running in CI on every push, so if I ever reintroduce that class of bug it fails the build instead of quietly making me look good.

Honest final numbers were 79.8% accuracy against a 39% baseline, and 0.93 macro ROC-AUC.”

Why interviewers love this story

It is a failure story where the resolution is *a worse number*. That is rare and it is credible, because nobody invents a story that makes their metric go down. It demonstrates:

- you understand evaluation deeply enough to catch a subtle bug;
- you were suspicious of your own good result — the single most valuable instinct in applied ML;
- you fixed the class, not the instance;
- you will tell an employer an inconvenient truth about your own work.

Two things to get right when telling it

1. **Verify the account against your own commit history.** If the actual cause differed, tell the real one — it is just as good a story and the risk of being caught out disappears.
2. **Never quote 80% alone.** Always “80% against a 39% majority-class baseline”. Alone, 80% sounds ordinary. With the baseline it is a doubling.

5.6.5 Other kinds of leakage — expect the follow-up

Kind	What it looks like
Preprocessing leakage	Yours. Fitting scalers/encoders/imputers before splitting.
Target leakage	A feature that is a proxy for the answer — e.g. including “cleaning fee” when cleaning fee is set as a fraction of nightly price.
Temporal leakage	Training on data from after the prediction point. Any time-ordered problem needs a time-based split, never a random one.
Group leakage	The same entity in both train and test — e.g. two listings from the same host, or the same patient’s two scans. Needs GroupKFold.
Duplicate leakage	Exact or near-duplicate rows appearing on both sides of the split.
Test-set overfitting	The slow leak: repeatedly checking the test set and tuning until it looks good. The test set stops being a test set the third time you look at it.

5.7 Results — the full table, not just the headline

Model	Test accuracy	Macro F1	Macro ROC-AUC
Majority-class baseline	0.390	0.187	0.500
Logistic regression	0.767	0.772	0.909
Random forest	0.792	0.797	0.926
XGBoost (selected)	0.798	0.802	0.930

The three things this table lets you say

1. **“I had a baseline.”** A majority-class baseline scores 39% accuracy and 0.5 ROC-AUC by definition — 0.5 is exactly random. Without that row, no number in the table means anything.
2. **“I tried simple before complex.”** Logistic regression at 0.767 versus XGBoost at 0.798 is a three-point gain for a large jump in complexity. Reporting that honestly is more impressive than only showing the winner — and if the deployment constraint had been different, logistic regression would have been the right call.
3. **“I selected on macro F1, not accuracy.”** Say why: macro F1 averages per-class F1 equally, so it does not let the model win by neglecting the smallest class. On a nearly balanced problem the two agree, but choosing the metric *before* looking is the discipline.

5.7.1 Where the errors are — the best detail in the project

The error structure

Errors sit almost entirely between *adjacent* tiers. Budget and Premium — the two non-adjacent classes — are confused in only **16 of 3,541** test listings: **0.5%**.

	pred. Budget	pred. Standard	pred. Premium	
actual Budget	correct	adjacent error	0.5% only	The model almost never makes an <i>absurd</i> mistake. Its errors cluster where the thresholds are arbitrary anyway — a \$205 listing called Budget is barely wrong. The sentence to land: “the errors are where the boundary is arbitrary, not where the model doesn’t understand the market.”
actual Standard	adjacent error	correct	adjacent error	
actual Premium	0.5% only	adjacent error	correct	

5.8 The application layer

Module	What it does
Price Advisor	Takes a listing’s attributes, returns the market tier, a recommended nightly band, a confidence figure, a revenue outlook, and the comparable neighbourhood listings that anchor the recommendation.
Market Explorer	Filterable view of the whole snapshot: live KPI band, price-tier map with per-listing hover, priciest-neighbourhood ranking, price distributions, distance-to-CBD price decay, room-type composition by tier, amenity premium calculations, per-neighbourhood deep dives.

What “confidence-scored” means — and what it does not

The confidence figure is the model’s **predicted class probability** — for tree ensembles, a normalised vote share across the ensemble.

What it is: a useful *relative* signal. A 0.92 prediction is more reliable than a 0.41 one, and the app treats a low-confidence result differently.

What it is not: a calibrated probability. “0.8” does not necessarily mean the model is right 80% of the time. Tree ensembles are typically over-confident in the middle of the range.

If asked how you’d fix that: Platt scaling or isotonic regression via `CalibratedClassifierCV`, fitted on a held-out calibration split, then verified with a reliability diagram and Brier score. Being able to say “I report a confidence but I have not formally calibrated it, and here is how I would” is a much stronger answer than claiming calibration you did not do.

Explainability — what you did, and what you would add

What the app does now: anchors every recommendation against comparable neighbourhood listings, so the host sees *why* the tier is plausible in market terms rather than being handed a number from a black box. The Market Explorer exposes the drivers directly — distance decay, amenity premiums, room-type composition.

What you would add: SHAP values for per-prediction attribution — “this listing was placed in Premium mainly because of distance to the beach and the pool flag”. Say it as a roadmap item, not as something you shipped.

The distinction to name: *global* explainability (which features matter overall — feature importances) versus *local* explainability (why *this* prediction — SHAP, LIME). Users need local; model developers need global.

5.9 Engineering around the model

- **Config-driven.** `config/config.yaml` drives the pipeline, so the thresholds, feature set and model choice are data rather than code.
- **Artifact versioning.** The saved model carries its own metrics, the snapshot date, the training timestamp and library versions, mirrored in `reports/metrics.json`. This is the cheap version of a model registry: you can always answer “which model produced this number, trained on what, with what library versions”.
- **Defensive feature detection.** The pipeline detects features that are completely unobserved in a given snapshot and excludes them — three columns were entirely empty in the June 2026 snapshot. Without this, a new snapshot crashes or, worse, imputes a constant into a feature the model relies on.
- **CI.** GitHub Actions runs the test suite on every push.
- **Reproducible data.** The raw file is downloaded by command rather than committed — the opposite choice to GridPulse, and worth being able to justify: the Airbnb file is large and licensed for redistribution differently, and the download is a single deterministic URL.

5.10 Limitations — volunteer these

- One city, one timestamp. The model should be *retrained* per snapshot, not reused — and the artifact records its snapshot date precisely so that misuse is detectable.
- Asking prices, not transactions. No occupancy or revenue signal.
- Tier thresholds are a modelling choice; listings near a boundary are inherently ambiguous.
- Confidence is uncalibrated (above).
- Decision support, not financial advice — stated in the app itself.

5.11 Question bank — Airbnb

Q. Why XGBoost rather than a neural network?

A. Tabular data with 24 mostly-engineered features and 17.7K rows. Gradient-boosted trees are the strongest default in that regime — they handle mixed types and non-linear interactions without much tuning, they don’t need feature scaling, and they train in seconds so I could iterate. A neural network would need more data, more tuning and more compute to reach parity, and would be harder to explain to the person using the app.

Q. What is ROC-AUC, in plain language?

A. Take one random positive example and one random negative one. ROC-AUC is the probability that the model scores the positive higher. 0.5 is random guessing; 1.0 is perfect. For a three-class problem I report the macro average over one-vs-rest AUCs. I use it alongside accuracy because it measures the quality of the *ranking* of the scores, independent of where you put the decision threshold.

Q. What’s the difference between macro and weighted F1?

A. Macro averages the per-class F1 scores equally, so a small class counts as much as a large one. Weighted averages them by class size, so a model can look good while failing the minority class. On a balanced problem they’re close; on an imbalanced one, macro is the honest choice and weighted is the flattering one.

Q. Your classes are balanced. What if they weren’t?

A. Three levers, in order of preference. First, change the metric — macro F1, per-class recall, precision-recall AUC rather than accuracy. Second, class weights, which most sklearn estimators and XGBoost support directly and which cost nothing. Third, resampling — SMOTE or undersampling — which I’d use

last, because synthetic minority samples must be generated *inside* the CV fold or you have reintroduced leakage, and it's an easy mistake to make.

Q. How do you know 79.8% is good?

A. Against a 39% majority-class baseline, it's roughly double. Against logistic regression at 76.7% it's a modest gain, which tells me most of the signal is linear and the tree model is picking up interaction effects at the margin. And the error structure matters more than the headline: 0.5% confusion between non-adjacent tiers means the model has genuinely learned the market ordering.

Q. The app gives a host a price. What if they follow it and lose money?

A. That's why the framing is a tier and a band with comparables rather than a single number, and why the app is explicit that it's decision support and not financial advice. The model predicts what the market *asks* for a listing like this — it has no occupancy or booking data, so it cannot tell you what maximises revenue. Being clear about the boundary of what the model knows is part of shipping it responsibly.

Q. How would you monitor this in production?

A. Three things. *Data drift* — compare the feature distribution of incoming listings to training, e.g. population stability index or a KS test per feature. *Prediction drift* — watch the tier mix; if Premium share doubles overnight, something upstream changed. *Performance* — which needs ground truth, and here that means retraining on the next snapshot and checking whether the previous model still holds up. Given the data is a periodic snapshot, scheduled retraining with a champion/challenger comparison is the natural design.

Q. What would you do differently if you started again?

A. Two things. I'd set up the Pipeline and the CV structure *first*, before any modelling — the leakage happened because I treated preprocessing as a step that happens before the real work, and it isn't. And I'd add a calibration split from the start, so the confidence number the app shows is a probability I can defend rather than a score I can only rank by.

6. Project 3 — SignSpeak: real-time sign language to speech

Repo: github.com/adhu-601/Hands-Signs-to-Speech **Live:** signspeak-asl.streamlit.app
Stack: Python, MediaPipe Tasks, scikit-learn (SVM), OpenCV, NumPy, Streamlit, streamlit-webrtc, gTTS
Licence: MIT

6.1 The one-liner

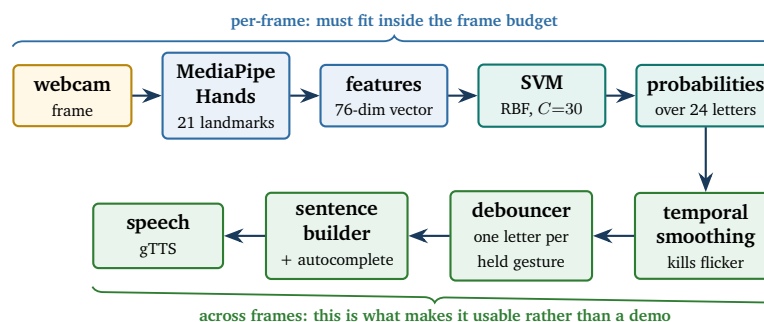
20 seconds

“SignSpeak reads ASL fingerspelling through a webcam and turns it into typed text and spoken words, live in the browser. Twenty-four static letters, a support vector machine over hand-landmark geometry, and enough smoothing and debouncing on top that it builds sentences rather than flickering between letters.”

60 seconds

“The interesting part is what it *isn't*. I started with a convolutional network classifying raw image crops of hands. It worked in my kitchen and fell apart everywhere else — different backgrounds, different lighting, different skin tones — and it was far too heavy to feel real-time in a browser. So I replaced the whole perception layer with MediaPipe Hands, which gives me 21 landmark coordinates, and built explicit geometric features on top: recentre on the wrist, scale by hand spread so it's position and size invariant, then add fingertip distances, per-finger curl ratios and finger directions in the palm's frame. Seventy-six features total. On those features a plain SVM with an RBF kernel gets 99.4% test accuracy at 13 milliseconds a frame and ships as a 2.3 megabyte file. The lesson was that the small model wasn't a compromise — the features were better, so the model didn't need to be big.”

6.2 The recognition pipeline — draw this



The half of the system that is not machine learning

The classifier is 99% accurate *per frame*. At 20 frames a second, that is still a wrong letter every few seconds, and a raw per-frame output is unusable — it flickers.

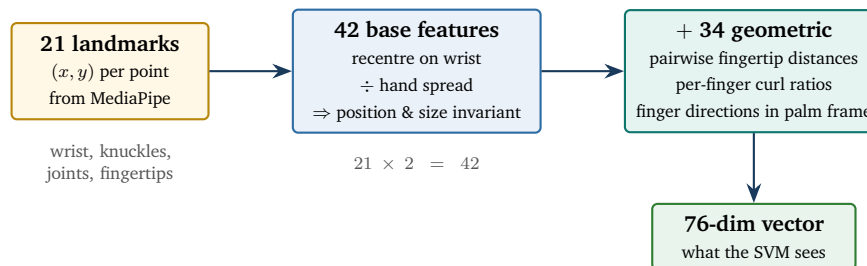
Three non-ML components make it a product:

- **Temporal smoothing** — average or vote the probabilities over a short window, so a single bad frame cannot emit a letter.
- **Debouncing** — borrowed from hardware button design. A held gesture should emit *one* letter, not sixty. The debouncer requires the smoothed prediction to be stable for a minimum dwell time, then locks out re-emission until the hand changes.

- **Autocomplete** — frequency-ranked suggestions from a 10,000-word dictionary, three at a time. This is the single biggest usability win: fingerspelling is slow, so finishing the word for the user is worth more than another point of accuracy.

This is the point to land: “most of the work in making a model usable happens after the model. The accuracy number was the easy part.”

6.3 Feature engineering — 21 landmarks to 76 dimensions



Transform	Why it is there
Recentre on the wrist	Removes <i>translation</i> . Where the hand is in the frame carries no information about which letter it is.
Divide by hand spread	Removes <i>scale</i> . A hand near the camera and the same hand far away must produce the same vector.
Pairwise fingertip distances	Encodes the relations that actually distinguish letters — how far the index is from the thumb is the whole difference between several handshapes.
Per-finger curl ratios	A single number per finger for “how bent is it”, which is exactly the axis ASL handshapes vary along.
Direction in the palm’s frame	Removes <i>rotation</i> dependence by expressing directions relative to the palm rather than the image.

Invariance — the concept behind all of it

Every one of those transforms removes a nuisance variable: translation, scale, rotation. What is left is only the information that distinguishes one letter from another.

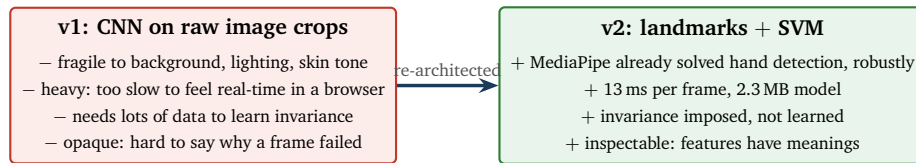
This is the classical alternative to what a deep network does. A CNN *learns* invariance from data, which requires a lot of data covering every position, scale and lighting condition. Hand engineering *imposes* invariance for free, which is why the small model wins here: I am not asking it to learn what I already know.

The measured effect: adding the 34 geometric descriptors moved the letter R from 0.953 to 0.977 F1 on the right hand and to a perfect 1.000 on the left. That is a feature-engineering win, not a modelling one — and being able to attribute the gain to the right cause is the point.

The augmentation subtlety — a genuinely good detail

Augmentation (small jitters, rotations) is applied to the **42 base features**, and the 34 geometric descriptors are then derived from the augmented values — *not* applied to the final 76-dim vector. Why it matters: if you perturbed the geometric features directly, you would create vectors that are internally inconsistent — a fingertip distance that does not correspond to any real hand pose. The model would learn to be robust to noise it will never see, and would not learn robustness to the noise it *will* see. Augmenting at the base and re-deriving guarantees every training sample is something the application could actually compute.

6.4 The pivot: CNN to landmarks



The judgement story — 45 seconds

“I started with a convolutional network on raw images. It worked, but it was fragile against backgrounds and lighting, and far too slow to feel real-time in a browser.

So I replaced it with something much simpler — extracting hand landmark coordinates with MediaPipe and running a lightweight classifier on those instead. Twenty-four ASL letters, over 99% test accuracy, and fast enough to actually run live on a free-tier server with no GPU.

The lesson I took was that the smaller model wasn’t a compromise, it was the correct answer — the features were better, so the model didn’t need to be big. I’m wary of reaching for the heaviest tool first.”

Why this story is worth telling to any employer

Almost every graduate CV says “deep learning”. Very few say “I replaced deep learning with something simpler and it was better”. That is an *engineering judgement* signal, and it is scarce.

The generalisable version: *“the right question isn’t which model is most powerful, it’s where the difficulty of the problem actually lives. Here it lived in perception, and MediaPipe had already solved that better than I could. Once perception was handled, the classification problem was easy — so using a big model for it would have been paying for capacity I didn’t need.”*

6.5 Model selection — accuracy is not the only axis

Model	Test acc.	Macro F1	CV macro F1	Latency
MLP (256, 128)	0.9909	0.9910	0.9956	0.6 ms
Random forest (400 trees)	0.9909	0.9915	0.9958	81.7 ms
Extra trees (400)	0.9909	0.9915	0.9975	103.7 ms
SVM (RBF, $C=30$) — selected	0.9943	0.9941	0.9976	13.4 ms

How to talk through this table — it is the best table you own

- **Accuracy alone would not decide this.** Three models are within a rounding error of each other. The deciding column is latency.
- **The tree ensembles are eliminated by latency, not accuracy.** 82–104 ms per frame is 10–12 frames a second before you have done anything else. It would not feel live.
- **The MLP is fastest but not most accurate,** and at 0.6 ms the extra speed buys nothing — I am already comfortably inside the frame budget at 13 ms. *Speed you cannot perceive is not worth accuracy you can.*
- **SVM wins on both accuracy and CV stability,** at a latency that is comfortably inside budget. It also trains in seconds and ships as a 2.3 MB file, which matters for deploying to a free-tier host.

The sentence: “I selected on the constraint that actually bound — per-frame latency — not on the metric that was easiest to report.”

6.6 Numbers and details

SignSpeak facts

Classes	24 static ASL letters
Excluded	J and Z — they require motion, and a single frame cannot represent them
Landmarks	21 per hand, from MediaPipe Hands (Tasks API)
Feature vector	76 dimensions (42 base + 34 geometric)
Training samples	4,412 landmark samples, $\sim 5\times$ augmentation
Split	stratified 80/20
Selected model	SVM, RBF kernel, $C = 30$
Test accuracy	0.9943
Macro F1	0.9941
CV macro F1	0.9976
Per-frame latency	13.4 ms
Model size	2.3 MB
Residual errors	U/V/R and M/N — genuinely similar handshapes
Autocomplete	top-3 suggestions from a 10,000-word frequency-ranked dictionary

Consistency check before you quote it

Your CV says **99.2%**. The repository reports **0.9943** test accuracy and **0.9941** macro F1. Decide which figure is current, quote that one everywhere, and update whichever document is stale. See §16.

If asked to explain a discrepancy: “that’s an earlier figure before the geometric features went in” is a perfectly good answer *if it is true*. Check first.

6.7 Product decisions worth mentioning

- **Explicit camera start.** The user presses a button rather than the page requesting webcam access on load. A small thing that says you thought about consent rather than about the demo.
- **Practice mode.** Shows a target letter with a handshape description and scores the attempt — turns a classifier into a teaching tool.
- **Alphabet reference.** All 24 letters with descriptions.
- **Live skeleton overlay and confidence gauges.** The user can see *why* the model is uncertain — because the hand is half out of frame — rather than just being told it failed. That is explainability for a vision system.
- **Deployment constraints handled.** opencv-python-headless (no GUI libs on a server), WebRTC with a snapshot fallback for restrictive networks, browser-side audio, no GPU required.

6.8 Limitations — volunteer these

- **Fingerspelling only, not ASL.** Real ASL is a full language with its own grammar, non-manual markers and two-handed signs. This spells words letter by letter. Be *scrupulous* about this distinction — overclaiming here is both technically wrong and disrespectful to the language and its users.
- **No motion letters.** J and Z are excluded because the model is per-frame. Handling them needs a temporal model — a small sequence model or an LSTM over a landmark window.
- **Single-hand, front-facing.** It assumes a reasonably posed hand in view.
- **Dataset scale and diversity.** 4,412 samples is modest, and I should be clear about who signed them — landmark features remove the skin-tone sensitivity a raw-image CNN has, which is a real fairness advantage, but hand size and signing style variation still matter.

6.9 Question bank — SignSpeak

Q. Why an SVM? Isn't that dated?

A. It's the right tool for the shape of this problem: 76 dense, well-engineered, continuous features, a few thousand samples, and 24 classes. An RBF-kernel SVM finds a non-linear boundary in that space very effectively, trains in seconds, and produces a small model. "Dated" is about fashion, not fit — and I did try the alternatives and reported them.

Q. What is the RBF kernel doing, and what is C ?

A. The kernel trick lets the SVM find a linear boundary in a higher-dimensional space without computing the coordinates in it. The RBF kernel measures similarity as a function of distance, so the effective boundary in the original space is smooth and non-linear. C is the regularisation trade-off: high C penalises misclassified training points heavily, giving a tighter boundary and more overfitting risk; low C allows more slack. $C = 30$ is fairly high, which is reasonable here because the classes are genuinely well separated in the engineered feature space. I chose it by cross-validation, not by hand.

Q. 99.4% sounds too good. Is it leaking?

A. Fair challenge, and it's the right instinct. The split is stratified 80/20 on samples. The honest risk is *group leakage* — if multiple augmented variants of the same original capture land on both sides of the split, or the same signer's near-identical frames do, the test set is easier than reality. The correct guard is to split by source capture, not by row. That's the limitation I'd name before anyone else does, and it's why the number I trust more is how it behaves live, which is noticeably harder than 99%.

Q. Why does it get U, V and R wrong?

A. Because they are genuinely similar handshapes — two extended fingers in slightly different configurations. The confusions are exactly where a human learner also struggles, which is a good sign: the model's errors are structured rather than random. The fix would be features that discriminate that specific axis better — finger crossing and inter-fingertip angle — rather than a bigger model.

Q. How would you add J and Z?

A. They need motion, so the unit of prediction has to change from a frame to a short window. I'd buffer the last N landmark vectors and either add engineered motion features — fingertip trajectory direction and curvature — to the same classifier, or use a small sequence model over the window. The architectural point is that the pipeline already produces a clean per-frame vector, so adding a temporal layer is an extension rather than a rewrite.

Q. How do you test a real-time vision system?

A. You separate what's testable from what isn't. The feature mathematics is pure and deterministic — given landmarks, assert the 76-dim vector, including the invariance properties: translate the input and the output must not change. The debouncer and sentence builder are pure logic over a sequence of predictions, so they're straightforwardly unit-testable with synthetic sequences. The model itself is tested by held-out metrics, and the camera path is the only part that needs manual verification. That's what the pytest suite covers.

7. Project 4 — Securing large files with hybrid cryptography

Stack: C++, OpenSSL (AES + RSA), POSIX/std threading

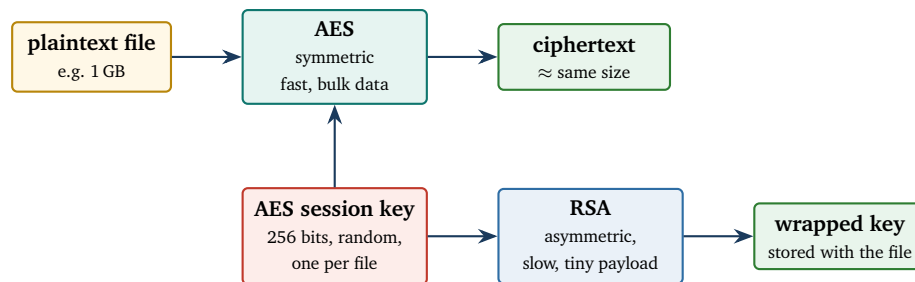
Result: under one minute per 1 GB, roughly $3\times$ faster than the single-threaded version. Related work shipped at Magic Software improved throughput $\sim 40\%$ in a commercial product.

7.1 The one-liner

25 seconds

“It’s a file encryption engine in C++ for large confidential files. It uses hybrid cryptography — AES for the file itself, RSA to protect the AES key — and it processes the file in independent chunks across threads, which is what brought a gigabyte under a minute, about three times faster than the serial version.”

7.2 Why hybrid — the concept you must have cold



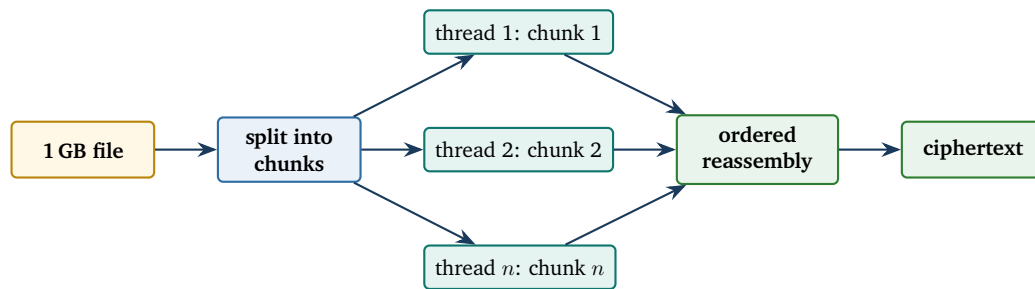
RSA never touches the file. It encrypts *only* the 256-bit key — which is the entire point of the design.

Say this and you have answered the question completely

- **Symmetric (AES)** — one shared key, very fast, hardware-accelerated on modern CPUs (AES-NI). Gigabytes per second. But: how do you get the key to the recipient?
- **Asymmetric (RSA)** — a public key anyone can encrypt to, a private key only the recipient holds. Solves key distribution. But it is orders of magnitude slower, and it can only encrypt a payload smaller than the modulus — a 2048-bit RSA key can encrypt about 200 bytes. *You cannot RSA-encrypt a gigabyte, and not just because it would be slow: it is structurally impossible.*
- **Hybrid** — use each for what it is good at. Generate a random AES session key, encrypt the file with AES, encrypt the session key with the recipient’s RSA public key, ship both.

And the punchline that shows you know the context: “this isn’t a clever trick I invented — it’s how TLS, PGP and essentially every real system works. The alternative isn’t a design choice, it’s a mistake.”

7.3 Parallelism — and the thing that makes it non-trivial



The correctness condition: chunks must be independently decryptable *and* re-assembled in the right order. Each chunk carries its own IV/nonce and its index. Get this wrong and you have built a very fast way to produce corrupt files.

Concept	What you should be able to say
IV / nonce	Initialisation vector: a per-message random value that makes the same plaintext encrypt to different ciphertext under the same key. Reusing an IV with the same key is a catastrophic break in CTR and GCM modes — it leaks the XOR of two plaintexts. So each chunk needs its own.
Mode of operation	AES encrypts 16-byte blocks; a mode says how to chain them. <i>ECB</i> encrypts each block independently and is broken — identical plaintext blocks produce identical ciphertext, so patterns survive. <i>CBC</i> chains blocks, which makes it inherently serial. <i>CTR</i> turns the block cipher into a stream cipher and is parallelisable by construction. <i>GCM</i> is CTR plus authentication.
Why CTR/GCM matters here	CBC cannot be parallelised within a stream because each block depends on the previous ciphertext block. CTR can, because each block's keystream depends only on the counter. <i>If you chunk the file, you are effectively choosing a parallel-friendly structure — say so.</i>
Authenticated encryption	Encryption hides content; it does not prevent tampering. AES-GCM (or encrypt-then-MAC) gives you integrity as well, so a modified ciphertext fails to decrypt rather than producing plausible garbage. This is the improvement to name if asked what you'd do differently.
Amdahl's law	Speedup is bounded by the serial fraction. $\sim 3\times$ rather than $n\times$ is expected: file I/O, chunking and reassembly are serial, and disk bandwidth eventually binds before CPU does. <i>Being able to explain why it is $3\times$ and not $8\times$ is better than claiming $8\times$.</i>
Thread safety	The OpenSSL context is not shared across threads — each thread gets its own cipher context. Shared mutable state was limited to the output assembly, which is where the ordering guarantee lives.

The question that separates “I used a library” from “I understand it”

“Why is ECB mode bad?” If you only remember one crypto answer, remember this one, because it is the most commonly asked.

“ECB encrypts each 16-byte block independently with the same key, so identical plaintext blocks produce identical ciphertext blocks. Structure in the plaintext survives into the ciphertext — the standard demonstration is encrypting a bitmap in ECB and still being able to see the picture. Any real mode uses an IV and chains or counters so that identical blocks encrypt differently.”

7.4 Question bank — cryptography

Q. Why not just use RSA for the whole file?

A. Two reasons, and the second is the important one. It would be thousands of times slower — but more fundamentally, RSA can only encrypt a message shorter than its modulus. A 2048-bit RSA key encrypts about 200 bytes. Encrypting a gigabyte isn't slow, it's impossible without splitting it into hundreds of

millions of RSA operations, which is not a design anyone would ship. Hybrid encryption exists precisely because of this.

Q. How do you choose the AES key size?

A. AES-256 for this. AES-128 is still considered secure and is faster, but for files at rest with a long lifetime the margin is worth the cost, and the cost is small because the key schedule is amortised across the whole file. The realistic threat to a file at rest is not brute force against AES — it's key management, which is why the interesting part of the design is how the session key is wrapped and stored.

Q. What if the private key is lost?

A. The data is gone. That is the correct property of the system and also its operational risk, which is why real deployments need key escrow or multi-recipient wrapping — encrypt the session key separately under several public keys so any authorised holder can open it. That's a natural extension of the design, since the session key is already independent of the bulk encryption.

Q. How would you make this production-grade?

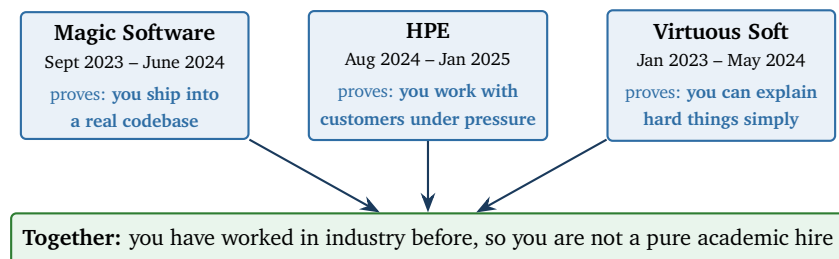
A. Four things. Use authenticated encryption (AES-GCM) so tampering is detected rather than silently decrypting to garbage. Use a vetted key-derivation function if any key comes from a password — Argon2 or scrypt, never a bare hash. Zeroise key material in memory after use, which in C++ means being careful that the compiler doesn't optimise the memset away. And move key storage to a proper keystore or HSM rather than a file next to the ciphertext.

Q. Did you write the crypto primitives yourself?

A. No, and that's deliberate — I used OpenSSL. Writing your own primitive is the classic way to introduce a side-channel or a subtle bias. The engineering was in the composition: the hybrid scheme, the chunking, the per-chunk IVs, thread-local contexts and the ordering guarantee on reassembly. *“Don't roll your own crypto” is one of the few pieces of advice in this field that is genuinely absolute.*

8. Experience — three roles, and what each one proves

Each role on your CV exists to prove something a project cannot. Know which is which, or you will tell the wrong story at the wrong moment.



8.1 Magic Software Enterprises — Software Developer (internship)

Sept 2023 – June 2024, Pune. C++, OOP, Magic XPA, OpenSSL. Six-person Agile team, commercially released product.

60 seconds

“I spent about ten months at Magic Software as a developer, in a six-person Agile team shipping into a commercially released product. I shipped five or so features — the one I’m proudest of was a large-file encryption capability that improved throughput by around 40%. But the part that shaped how I work was the bug queue. I fixed thirty-plus customer-reported issues in a large C++ codebase I hadn’t written, and almost none of them arrived as a stack trace. They arrived as ‘it’s slow on Tuesdays’ or ‘the report looks wrong’. So the job was really reproducing the problem first, then finding it, then fixing it without breaking three other things — and writing it down afterwards so the next person didn’t repeat the investigation. I mention it because joining a small team means inheriting a codebase, and I’ve done that before.”

Have a method for “how do you debug code you didn’t write?”

This is one of the most commonly asked questions for anyone with a maintenance background, and a structured answer beats an anecdote:

1. **Reproduce it first.** A bug you cannot reproduce is a bug you cannot verify you have fixed. Most of the effort in a vague report goes here — turning “it’s slow on Tuesdays” into a deterministic case.
2. **Bisect, don’t stare.** Narrow to the smallest failing input and the smallest span of code. Version history is a bisection tool too — if it worked last release, the change set is finite.
3. **Form a falsifiable hypothesis.** “I think it’s X” followed by “and if it is, then Y must also be true” — then check Y. Guess-and-patch is how you fix the symptom and keep the bug.
4. **Write the failing test before the fix.** Otherwise you don’t know the fix worked, and there’s nothing stopping it regressing.
5. **Write down what you found.** The investigation is usually worth more than the patch, and it is the part that evaporates if nobody records it.

Likely probe	Where to go
“Tell me about the encryption feature.”	Straight into §7. This is the bridge between the job and the project — make the link explicit.
“What is Magic XPA?”	A low-code/rapid application development platform from Magic Software; the C++ work integrated with it. Say plainly that it’s a proprietary platform and that the transferable part was the C++ and the process, not the tool.
“What does Agile mean at your team’s scale?”	Be concrete, not doctrinaire: sprint length, standups, how work was estimated, how a bug got prioritised against a feature. Avoid reciting the manifesto.
“Give me a bug you’re proud of fixing.”	Pick one with a non-obvious cause. Use the five-step method above as the narrative spine.
“How did you measure the 40%?”	Have the answer: what was the benchmark, what file sizes, what hardware, was it wall-clock or CPU time, and was it measured more than once.

8.2 Hewlett Packard Enterprise — Technical Consultant

Aug 2024 – Jan 2025, Pune. Enterprise cloud and storage. 20+ customers, 90%+ of tickets within SLA, ~30% reduction in average resolution time.

45 seconds

“I was a technical consultant at HPE, troubleshooting hardware, storage and hybrid-cloud issues across Linux and Windows for twenty-plus enterprise customers. I closed over 90% of assigned tickets within target and brought average resolution time down around 30%. What it taught me was mostly about working with someone who is frustrated and needs the problem *understood* before they need it solved. And it taught me to translate — the person on the call is often not the engineer, and telling them the truth in a form they can act on is a separate skill from finding the answer.”

Likely probe	Answer
“What’s an SLA?”	Service Level Agreement — a contractual commitment on response and resolution time, usually tiered by severity. P1 is production-down and has the tightest clock. Missing one has commercial consequences, which is why the number is tracked.
“How did you reduce resolution time 30%?”	Be specific about mechanism, not effort: better triage on intake, reusable diagnostics for recurring classes of issue, documenting fixes so the second occurrence was fast. <i>If the number came from a team dashboard rather than your own measurement, say so — attributing a team metric entirely to yourself is a credibility risk.</i>
“Tell me about a difficult customer.”	Use the arc: they were angry → I acknowledged the impact before diagnosing → I gave them a next update time rather than a promise I couldn’t keep → outcome. Never disparage the customer.
“Why did you leave after six months?”	Straightforward and unapologetic: “I’d been accepted into the masters at Sydney and moved to Australia. It was a planned transition, not a departure.”

The transferable claim from HPE

Do not sell this role as “support”. Sell it as: “*I have spent six months being the person who has to explain a technical failure to someone who is affected by it and doesn’t share my vocabulary. That’s most of what makes an engineer useful to a business.*”

8.3 Virtuous Soft Solutions — IT Instructor

Jan 2023 – May 2024, Pune, part-time. Python, C++, UI/UX (Figma & HCI). 50+ learners, 15+ mentored projects. Also Design Head for university committees.

40 seconds

“For about a year and a half I taught Python and C++ part-time — fifty-plus learners and mentoring fifteen-odd projects end to end, from wireframe to working build. It’s the reason I write documentation without being asked, honestly. When you’ve had to explain a pointer to somebody four different ways, you stop assuming your first explanation was clear.

It also taught me UI/UX properly, because I was teaching it — Figma, wireframing, basic HCI principles. That’s why I built the interfaces on my own projects rather than treating the last mile as somebody else’s problem.”

Expect to be asked to demonstrate this live

The natural follow-up is “**explain [some technical concept] to me as if I’m not technical.**” Practise two in advance:

Overfitting: “It’s memorising the practice exam instead of learning the subject. You look brilliant on questions you’ve seen and hopeless on new ones. The whole discipline of holding data back is about making sure you’re measuring understanding rather than memory.”

An API: “It’s a waiter. You don’t go into the kitchen and you don’t need to know how the kitchen works — you have an agreed list of things you can ask for and an agreed way to ask. The menu is the contract, and things break when the kitchen changes the menu without telling you.”

The trap is condescension. Simplify the vocabulary, never the listener.

Why teaching is a genuine differentiator, not filler

Most graduate candidates have never had to make themselves understood to someone who could not follow them. You have, fifty times. That predicts three things employers actually value: documentation that exists, code reviews that are useful, and the ability to explain a model’s output to the person who has to act on it.

Tie it to your work explicitly: “it’s the same reason my Airbnb app returns a confidence-scored recommendation with comparables instead of a number — a result somebody can’t interpret isn’t much use to them, however correct it is.”

9. Research and patent

9.1 IEEE ICBDS 2023 — Security, Privacy and Scalability in Blockchain and IoT

ieeexplore.ieee.org/document/10346378 — IEEE International Conference on Blockchain and Distributed Systems Security (ICBDS) 2023. Cited by 7.

The three things you must be able to say about your own paper

1. **The argument in one sentence.** Not the topic — the claim. Something of the form: “blockchain is often proposed as a security layer for IoT, but the three properties you want — security, privacy and scalability — trade off against each other, and here is how.”
2. **Your specific contribution.** Which sections, which experiments, which analysis. On a multi-author paper, be exact. “I did the literature synthesis and wrote sections 3 and 4” is a strong answer; a vague claim to the whole paper is a weak one.
3. **One limitation.** Every paper has one. Naming it yourself is the single clearest signal that you understand the work rather than remember it.

Action item: re-read the paper before any interview where it appears on the CV. Two years is long enough to have lost the detail, and being unable to discuss your own publication is far worse than not having one.

Likely probe	Have ready
“What’s the paper about?”	The one-sentence claim, then the trade-off triangle: security, privacy and scalability in blockchain-enabled IoT — no configuration optimises all three.
“Why is blockchain hard for IoT?”	IoT devices are constrained — limited compute, memory, power and intermittent connectivity. Consensus mechanisms are expensive. Ledgers grow monotonically, which is a problem for a device with megabytes of storage. And a public immutable ledger is in direct tension with privacy, because you cannot delete.
“Cited by 7 — do you know by whom?”	Worth checking before an interview. Knowing who built on your work is a strong detail.
“Would you write it differently now?”	Yes, and say how. Two years of a masters in data science should have changed your view of something in it.
“Was this coursework or research?”	Answer plainly. Undergraduate research that reached a conference is impressive on its own terms; there’s no need to inflate it.

9.2 Granted patent — IoT simulation and cloud integration system

Granted, South Africa, 2023.

How to talk about a patent honestly and effectively

What to say: what the system does, what problem it solves, and what the claim covers. A patent claim is a precise boundary — knowing yours is the whole test of whether you understand the filing.

What not to overstate: South African patents are granted without substantive examination of novelty in the way that, say, a US or EPO grant involves. A well-informed interviewer may know this. The correct posture is matter-of-fact: it is a real granted patent, it demonstrates you took an idea through a formal filing process as an undergraduate, and you are not going to claim it is more than it is.

The framing that works: *“It came out of the same work as the IEEE paper — IoT devices generating data that needs to reach a cloud backend, and the middleware that sits between them. Filing it taught me to describe a system precisely enough that the description alone defines it, which is a useful discipline for specs generally.”*

9.3 Top Project Award — Project Management in IT, University of Sydney

Recognised by TPG and Optus, 2025. [\[certificate\]](#)

Probe	Have ready
“What was the project?”	The problem, the deliverable, the team size, your role, and what the industry judges said. Be specific — an award with no story attached reads as decoration.
“Why did it win?”	Have a view. Usually: it solved a real problem, was scoped realistically, and was communicated well. If the last one is true, it connects directly to your teaching background.
“What was your role?”	Precise, not inflated.

10. Education

10.1 Master of Computer Science (Data Science and AI) — University of Sydney

Feb 2025 – Dec 2026.

Probe	Have ready
“What have you studied?”	Name the units that produced work you can show. COMP5339 Data Engineering produced GridPulse; the research study behind the Airbnb project came out of your coursework; Project Management in IT produced the award. <i>A unit that produced an artifact is worth ten that didn't.</i>
“What’s your favourite subject and why?”	Have an answer, and make it a judgement rather than a preference: what you found hardest, or what changed how you think.
“What’s your capstone / thesis?”	Be ready with the current state, even if it is early. “I haven’t chosen yet, and here’s what I’m drawn to” is fine; a blank is not.
“What’s your WAM/GPA?”	Know it. If you’d rather lead with the undergraduate 9.4/10, do — but do not appear evasive.
“Are you doing this full time?”	Yes, finishing December 2026, with your availability as stated. See below.

10.2 Bachelor of Computer Engineering — VIIT Pune

Feb 2020 – May 2024. CGPA 9.4/10.

If an Australian interviewer is unfamiliar with the Indian grading scale, translate it once, briefly: “9.4 out of 10 — roughly a high distinction average.” Do not labour it.

10.3 Availability and work rights — say this cleanly, once

The availability answer

“I’m based in Sydney. I can start part-time from September 2026 alongside my final semester, and full-time from December 2026 when I finish. I’m eligible for the Temporary Graduate visa, subclass 485, with full work rights.”

Three rules for this conversation

1. **Volunteer it early** if the role is advertised as immediate-start. Discovering a timing mismatch at offer stage wastes everyone’s time and you lose the goodwill you built.
2. **Do not apologise for it.** It is a fact about a calendar, not a deficiency. Deliver it flat and move on.
3. **Do not over-explain the visa.** One sentence. If they need detail, they will ask, and HR will confirm it anyway. Anxious over-explanation reads as a problem where there isn’t one.

11. The skills audit — every claim on the CV, graded

This is the section that prevents the worst outcome in an interview: being caught out on a word you put on your own CV. For each claim, know which column you are in.

SHIPPED used in a live project claim freely, expect depth	USED built with it, not shipped claim with the context	STUDIED understand, not applied say “I’ve studied it, not shipped it”
--	---	--

11.1 AI and LLMs

CV claim	Honest grade	What to say
Prompt engineering	Used	“I’ve used LLMs as a tool and worked with prompting patterns, but I haven’t shipped an LLM feature into production. The evaluation discipline I have from the ML side is the part that transfers.”
Structured output & evaluation	Studied/Used	Know the mechanism (schema-constrained decoding, function/tool calling, JSON mode) and why it beats parsing free text. Say clearly what you have and haven’t built.
Retrieval over document collections (RAG)	Studied	Do not imply you have shipped a RAG system. Say: “I understand the architecture and can describe it, but what I’ve actually built is the data engineering around unreliable sources and the testing discipline. RAG-specific patterns would be my steepest learning curve.”
Human-in-the-loop review design	Used	Real: the Airbnb app is decision support with confidence and comparables; SignSpeak surfaces model uncertainty to the user. Frame it through those.
NLP / Generative AI	Studied	Coursework-level. Be plain about it.

The honesty move, and why it makes you *more* hireable

1. **Say so plainly.** “I haven’t built that.” No hedging.
2. **Name the adjacent thing you do have.** “I’ve done the equivalent with X.”
3. **Show the instinct.** “Here’s how I’d approach finding out.”

Say it like this

“I haven’t built a production RAG system — what I’ve built is the data engineering around unreliable sources and the testing discipline. So I’d expect the LLM-specific patterns to be the steepest part of the learning curve for me, and the pipeline, testing and data-quality side to be where I’m useful from the first week.”

Interviewers are actively testing whether you will bluff. Admitting one gap cleanly makes every other claim you make more credible. Bluffing one makes all of them suspect.

11.2 ML engineering

CV claim	Grade	Where it is defended
Python, pandas, NumPy	Shipped	All three live projects.
scikit-learn	Shipped	Pipeline, CV, SVM, model selection — §5, §6.
XGBoost	Shipped	§5.7. Know what gradient boosting is (§12).
Leakage-safe pipelines	Shipped	§5.6. Your strongest ML claim — it is backed by a failure story.
Class imbalance	Studied	Your dataset was balanced by design. Say so, then give the three levers (metric, class weights, resampling inside the fold).
Model evaluation, ROC-AUC	Shipped	§5.7.
Calibration	Studied	Be careful here. You report confidence; you did not calibrate it. Say exactly that, and how you would.
Confidence scoring	Shipped	The Airbnb app and SignSpeak’s gauges.

11.3 Data engineering

CV claim	Grade	Where it is defended
SQL	Shipped	dbt models in GridPulse. Expect a live SQL question — see §12.8.
dbt, Dagster, DuckDB	Shipped	§4, and you can justify each choice against alternatives.
PostgreSQL	Used	Be honest about depth. You can discuss OLTP vs OLAP; you have not run a production Postgres.
ETL/ELT	Shipped	And you can say which one GridPulse is and why.
REST API integration	Shipped	§4.4 — retry, backoff, rate limits, caching.
Star-schema modelling	Shipped	§4.5, including grain and additivity.
Data-quality testing, lineage	Shipped	§4.7, §4.6. This is your strongest data claim.

11.4 Production and cloud — the one to be most careful with

CV claim	Grade	What to say
Git	Shipped	Know branching, rebase vs merge, what a PR review is for.
CI/CD	Shipped	GitHub Actions on the Airbnb project. Know what runs, and what should fail a build.
pytest	Shipped	All three projects. Know fixtures, parametrisation, and unit vs integration vs data tests.
AWS (EC2, S3, Lambda, RDS, IAM) — “foundational”	Studied	The word “foundational” is load-bearing and you must honour it. Know what each service is for and when you’d reach for it. Do <i>not</i> claim to have run production infrastructure. If pushed: “I’ve studied the core services and understand the model — I haven’t operated AWS in production, and I’d expect that to be a learning curve.”
Linux	Shipped	Six months at HPE across Linux and Windows. Real.
C++, OOP, design patterns	Shipped	Magic Software plus the crypto engine.
Streamlit	Shipped	Three deployed apps.

The AWS trap

“AWS” on a CV invites “walk me through how you’d deploy this on AWS”. You should be able to sketch it — S3 for raw data, a Lambda or scheduled container for ingestion, RDS or a warehouse for the marts, IAM roles for least privilege, CloudWatch for logs — while being explicit that this is design knowledge rather than operational experience.

The failure mode is claiming operational depth and then not knowing what a security group is. The successful move is a confident design answer with an honest boundary attached.

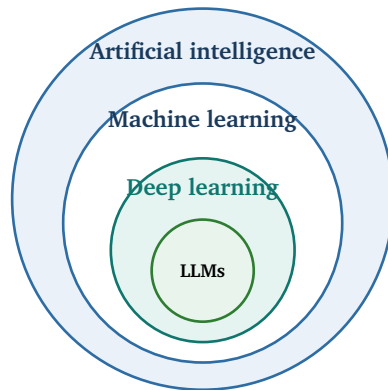
11.5 Product, UI/UX, security

CV claim	Grade	Notes
Figma, HCI, wireframing	Shipped	Taught it for 18 months and used it on your own projects. Genuine and unusual for an engineering candidate — lead with it when the role touches product.
Decision-support interface design	Shipped	The Airbnb advisor is the artifact.
Applied cryptography (AES, RSA, OpenSSL)	Shipped	§7. Strong, but expect ECB and IV-reuse questions.
Access control	Studied	Know the concepts — least privilege, RBAC, separation of duties. Don’t overclaim.
Handling confidential data	Used	HPE (enterprise customer data), plus the crypto work. Frame it as a habit, not a certification.
Technical writing	Shipped	The IEEE paper, project documentation, teaching materials.

12. Concept refreshers — the fundamentals under your CV

This section is the theory you are expected to hold because of what you wrote down. It is organised by the question you will be asked, not by textbook order.

12.1 The nesting: AI, ML, deep learning, LLMs



AI — any system doing something that looks intelligent, including pure rules.

ML — systems that learn the rules from data rather than being given them.

Deep learning — ML using many-layered neural networks.

LLMs — a specific deep-learning architecture (transformers) trained on text.

The distinction that matters in an interview

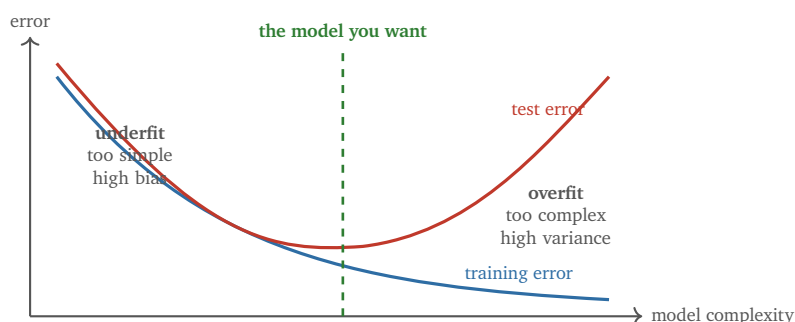
Not everything worth building is ML, and saying so is a strength. GridPulse contains no model at all — it is rules, validation and orchestration. SignSpeak's hardest problems were latency and debouncing, not classification.

"A rule you can write down is better than a model you have to trust, whenever the rule exists." Say that when someone asks where you'd apply AI, and you will sound like an engineer rather than an enthusiast.

12.2 Supervised, unsupervised, reinforcement

Kind	Definition	Yours
Supervised	Labelled examples: inputs paired with the right answer. Split into <i>classification</i> (discrete label) and <i>regression</i> (continuous value).	Both your ML projects. Both classification.
Unsupervised	No labels; find structure. Clustering, dimensionality reduction, anomaly detection.	None shipped — say so.
Reinforcement	An agent takes actions in an environment and learns from reward.	None. Don't claim it.
Self-supervised	Labels derived from the data itself — e.g. predict the next token. How LLMs are pre-trained.	Conceptual only.

12.3 Overfitting, underfitting, and the bias–variance trade-off



Term	Definition
Overfitting	The model has learned noise specific to the training data. Symptom: training error much lower than test error.
Underfitting	The model is too simple to capture the real pattern. Symptom: both errors high.
Bias	Error from wrong assumptions — a straight line trying to fit a curve.
Variance	Error from sensitivity to the particular training sample — change the data slightly and the model changes a lot.
Regularisation	Deliberately constraining the model to reduce variance: L1/L2 penalties, tree depth limits, dropout, early stopping.

How to answer “how do you know your model is overfitting?”

“I compare training and validation error. A large and growing gap is the signature. But the subtler case is the one I actually hit — if the whole evaluation is inflated by leakage, train and test both look fine and there is no gap to notice. That’s why I care as much about how the split was constructed as about the numbers it produces.”

That answer connects theory to your own story, which is exactly what an interviewer is hoping for.

12.4 The confusion matrix and the metrics that come from it

	predicted positive	predicted negative
actual positive	TP true positive	FN false negative (a miss)
actual negative	FP false positive (a false alarm)	TN true negative

Precision = $\frac{TP}{TP + FP}$ — of what I flagged, how much was right?

Recall = $\frac{TP}{TP + FN}$ — of what was there, how much did I find?

F_1 = harmonic mean of the two.

The question underneath every metric question: which error is worse?

Precision and recall trade off, and **which one you optimise is a business decision, not a technical one.**

- **Fraud detection** — recall matters; missing a fraud costs more than reviewing a clean transaction.
- **Spam filtering** — precision matters; a legitimate email in the spam folder is worse than a spam email in the inbox.
- **Medical screening** — recall, overwhelmingly, with a second-stage test for precision.

If asked to pick a metric, ask what the two errors cost before answering. Candidates who name a metric immediately have not understood the question.

Why F1 and not accuracy? Accuracy is useless under imbalance — a model that predicts “not fraud” every time is 99.9% accurate and worthless. F1 cannot be gamed that way.

Metric	Definition and when to use it
Accuracy	Fraction correct. Fine on balanced data, dangerous on imbalanced.
Precision	Of predicted positives, how many are correct. Use when false alarms are costly.
Recall (sensitivity)	Of actual positives, how many were caught. Use when misses are costly.
F_1	Harmonic mean of precision and recall — punishes a model that is good at one and bad at the other.
Macro F_1	Per-class F_1 averaged equally. The honest multi-class metric.
ROC-AUC	Probability a random positive scores higher than a random negative. Threshold-independent. 0.5 is random.
PR-AUC	Precision–recall AUC. Better than ROC-AUC when positives are rare, because ROC-AUC can look good while precision is terrible.
Log loss / Brier	Penalise confident wrong predictions. The metrics that care about <i>calibration</i> .

12.5 The model families, one line each

Family	One-line description you should be able to give
Linear / logistic regression	Weighted sum of features, passed through a function. Fast, interpretable, the right first thing to try, and a real baseline.
Decision tree	A sequence of yes/no splits. Perfectly interpretable, and alone, prone to overfitting.
Random forest	Many trees on random subsets of rows and features, votes averaged. <i>Bagging</i> — reduces variance.
Gradient boosting (XGBoost)	Trees built <i>sequentially</i> , each correcting the previous one's errors. <i>Boosting</i> — reduces bias. Usually the strongest option on tabular data.
SVM	Finds the boundary with the widest margin between classes; the kernel trick makes it non-linear. Strong on modest data with good features.
k -nearest neighbours	Predict by looking at the closest training examples. No training, slow prediction, a useful sanity check.
Neural networks	Layers of weighted sums and non-linearities. Dominant for images, audio and text; usually unnecessary for tabular data.

Bagging vs boosting — the classic question

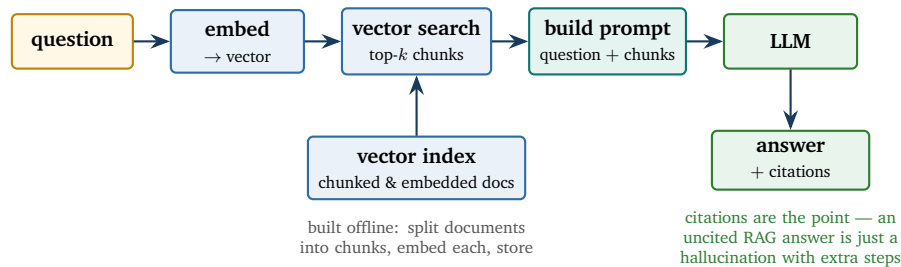
Bagging (random forest): many models trained *in parallel* on random subsets, then averaged. Each is a full-strength model; averaging cancels their individual errors. Attacks *variance*.

Boosting (XGBoost): many weak models trained *in sequence*, each fitted to the residual errors of the ensemble so far. Attacks *bias*.

The practical consequence: boosting usually wins on accuracy but is more sensitive to hyperparameters and can overfit if you let it run too long — which is why early stopping on a validation set is standard. Bagging is harder to get wrong.

12.6 LLMs — enough to be credible without overclaiming

Term	Definition
Token	A chunk of text, roughly 3–4 characters of English. Models read and write tokens, not words. Pricing and context limits are counted in tokens.
Context window	The maximum tokens the model can attend to at once — prompt plus output. Everything the model “knows about your situation” must fit inside it.
Next-token prediction	What the model actually does: given the text so far, produce a probability distribution over the next token, sample one, append, repeat. Everything else is a consequence of this.
Pre-training	Learning language from an enormous corpus, self-supervised.
Fine-tuning	Further training on a specific dataset to adapt behaviour or style.
Temperature	How randomly the next token is sampled. 0 is near-deterministic (but not exactly — floating-point and batching make true determinism hard to guarantee); higher is more varied.
Hallucination	Fluent, confident, false output. <i>Not a bug to be patched</i> — a direct consequence of a model that generates plausible continuations rather than retrieving facts.
Structured output	Constraining generation to a schema (JSON mode, function/tool calling) so the result can be parsed reliably instead of scraped from prose.
Embedding	A vector representing meaning, such that similar texts are close together. The basis of semantic search.
RAG	Retrieval-Augmented Generation — retrieve relevant documents first, put them in the prompt, and have the model answer from them.
Agent	An LLM in a loop with tools, deciding what to call next. Powerful and hard to make reliable, because errors compound across steps.



Prompting vs RAG vs fine-tuning — know when each is right

- **Prompting** — when the model already knows how to do the task and just needs instruction. Cheapest, fastest to iterate. Always try this first.
- **RAG** — when the model needs *facts it doesn't have*: your documents, current data, private knowledge. Solves knowledge, not behaviour.
- **Fine-tuning** — when the model needs to *behave differently*: a consistent format, a tone, a specialised task. Solves behaviour, not knowledge. Expensive, and it goes stale.

The mistake to name: people reach for fine-tuning when they have a knowledge problem. If the answer changes when your documents change, you need retrieval, not training.

How you would evaluate an LLM system — the answer that shows maturity

This is where your background is genuinely relevant, so say so:

“The same way I’d evaluate anything else: with a held-out set of cases where I know the right answer. The difference is that the output isn’t a class label, so you need graded criteria — is it factually supported by the retrieved context, does it cite correctly, does it decline when it should. I’d build a fixture set of known-correct cases as a regression suite, so when the prompt or the model changes I find out immediately whether the answers moved. That’s the same instinct as the 39 data-quality tests — if you can’t detect a regression automatically, you’ll ship one.”

Be scrupulous about the boundary

You have not shipped an LLM system. Say that. The credible position is: *“I’ve studied the architecture and I’ve built the surrounding disciplines — ingestion from unreliable sources, evaluation, testing, structured output for downstream consumers. The LLM-specific patterns are where I’d expect to learn fastest and where I’d want mentoring early.”*

12.7 Data engineering vocabulary

Term	Definition
OLTP vs OLAP	Transactional (many small reads/writes, row-oriented, Postgres) vs analytical (few large scans and aggregations, column-oriented, DuckDB/BigQuery). Different shapes for different jobs.
Columnar storage	Storing values of one column together. Makes “sum this column over a billion rows” cheap, because you read only that column.
Data warehouse	Structured, modelled, query-ready data.
Data lake	Raw files in cheap storage, schema applied on read. <i>Lakehouse</i> is the hybrid.
Idempotency	Running it twice produces the same result as running it once. The property that makes retries and backfills safe.
Full refresh vs incremental	Rebuild everything, versus process only what’s new. Full refresh is simpler and correct by construction; incremental is necessary at scale and is where the subtle bugs live.
Slowly changing dimension	How you handle a dimension attribute that changes over time — overwrite (Type 1) or keep history with validity dates (Type 2). Type 2 is what you need for auditability.
Data contract	An explicit, enforced agreement about the schema and semantics a producer will supply. Tests are the enforcement mechanism.
Freshness	How old the newest data is. A pipeline can pass every correctness test and still be useless if it’s three days stale — so freshness needs its own check.
Partitioning	Splitting a table by a key (usually date) so queries and rebuilds touch only what they need.

12.8 SQL — expect to write some

The five things you are most likely to be asked to write

1. **A join with an aggregate.** “Total energy per region” — join fact to dimension, GROUP BY, SUM. Know that WHERE filters rows *before* grouping and HAVING filters groups *after*.
2. **Join types.** INNER keeps only matches; LEFT keeps all of the left side with nulls where there’s no match. **The classic bug:** a LEFT JOIN followed by a WHERE condition on the right table silently turns it back into an inner join.
3. **A window function.** ROW_NUMBER() OVER (PARTITION BY region ORDER BY energy DESC) to get the top facility per region. This is the single most commonly tested SQL concept beyond joins — *a window function aggregates without collapsing rows*.
4. **Deduplication.** ROW_NUMBER() partitioned by the key, ordered by recency, then keep row 1.
5. **Finding gaps.** A date spine LEFT JOINed to your data, looking for nulls — exactly the shape of the “no gaps in the five-minute series” test in GridPulse.

The NULL trap

NULL = NULL is not true — it’s unknown. COUNT(*) counts rows including nulls; COUNT(col) skips nulls. NOT IN with a NULL in the subquery returns nothing at all. These produce silently wrong answers rather than errors, which puts them squarely in your “quietly wrong” theme — worth saying so if it comes up.

12.9 Software engineering vocabulary

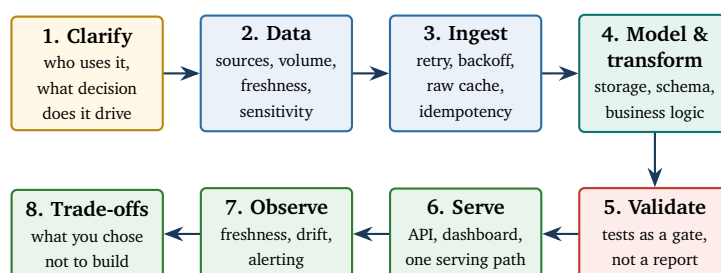
Term	Definition
Unit / integration / end-to-end	One function in isolation / several components together / the whole system as a user would use it. Fast and many at the bottom, slow and few at the top.
Fixture	Known input data set up for a test.
Mocking	Replacing a dependency (an API, a clock) with a controllable stand-in so the test is deterministic.
Regression test	A test that pins behaviour you already have, so a future change can't silently alter it. <i>The category that matters most in a system where correctness is the product.</i>
CI / CD	Continuous integration: run tests automatically on every push. Continuous delivery/deployment: ship automatically when they pass.
Code review / PR	Changes proposed on a branch and read by someone else before merging.
Technical debt	A deliberate shortcut with an ongoing cost. <i>Deliberate</i> is the operative word — an accident isn't debt, it's a mistake.
Observability	Logs, metrics, traces — being able to tell what a running system is doing.

12.10 Security and privacy vocabulary

Term	Definition
At rest vs in transit	Encrypting stored data vs encrypting data moving over a network (TLS). You need both, and they're different mechanisms.
Least privilege	Every identity gets the minimum access it needs. The default posture.
RBAC	Role-based access control — permissions attach to roles, users get roles.
PII	Personally identifiable information. Triggers legal obligations, not just good practice.
Anonymisation vs pseudonymisation	Irreversibly removing identity vs replacing it with a key you still hold. Only the first is genuinely anonymous, and true anonymisation is harder than it looks.
Audit log	An append-only record of who did what and when. Distinct from application logs, and usually a compliance requirement.
Privacy Act 1988 / APPs	Australia's governing framework. You are not expected to know it in detail — you are expected to know it exists and constrains design.

12.11 The system-design answer — a reusable template

If you are asked to design something — “how would you build X?” — use this shape. It works for almost any data or ML system and it stops you from diving into implementation too early.



Three rules for design questions

- 1. Ask before you answer.** Scale, freshness, who consumes it, how wrong it's allowed to be. A candidate who designs before asking has failed the main test.
- 2. Name the trade-off out loud at every choice.** “I'd start with X because Y; the cost is Z, and I'd revisit it when W.” Interviewers are scoring reasoning, not the answer.
- 3. Say what you would not build first.** Scoping is the scarcest skill. “I'd ship the pipeline and the validation before any modelling, because if the data's wrong the model is irrelevant” is a

genuinely senior answer — and it's yours honestly, because it's what you did on GridPulse.

13. Behavioural questions — your stories, mapped

13.1 STAR, and the ratio that matters

STAR

Situation — what was going on. **Task** — what you needed to do. **Action** — what you specifically did. **Result** — what happened, ideally with a number.
Aim for roughly 15% / 15% / 50% / 20%. Most candidates spend two-thirds on Situation and run out of time before Action, which is the only part being scored.

13.2 The story matrix — one table to cover the whole behavioural round

You have six stories. Almost every behavioural question maps to one of them. Learn the mapping, not new stories.

Question	Story to use	Section
Tell me about a time you failed / made a mistake	Airbnb leakage — the model you had to un-ship	§5.6
Tell me about a difficult technical problem	GridPulse silent load — no error, wrong number	§4.8
A time you found a bug nobody else noticed	GridPulse silent load	§4.8
A time you had to change your approach	SignSpeak CNN pivot	§6.4
A time you chose the simpler option	SignSpeak pivot	§6.4
A time you worked on something you didn't write	Magic Software bug queue	§8.1
A time you dealt with a difficult stakeholder	HPE — frustrated customer	§8.2
A time you explained something technical to a non-technical person	Teaching , or the Airbnb app's design	§8.3
A time you took initiative / went beyond the brief	Turning coursework (GridPulse, Airbnb) into packaged, tested, deployed products	§4
A time you disagreed with someone	Use a technical disagreement with evidence — and land on how it resolved, not on being right	—
A time you had to prioritise under pressure	HPE ticket triage, or the roadmap decisions on GridPulse	§8.2
What are you most proud of?	GridPulse or the leakage fix. <i>Pick the one that fits the role.</i>	—
What's your biggest weakness?	See below — do this properly	—

The weakness question, done properly

Do not use a disguised strength (“I care too much”). Interviewers have heard it and it reads as evasion. Use a real one with an active remedy:

Say it like this

“I’ve historically been too quick to trust a good result. The Airbnb leakage is exactly that — the score jumped, and my first reaction was pleasure rather than suspicion. What I’ve changed is procedural rather than attitudinal, because attitude doesn’t hold under deadline: I now build the evaluation structure before the model, and I write tests that fail if I reintroduce the mistake. I don’t trust myself to remember to be careful.”

That answer is real, it is already fixed, and it doubles as a demonstration of the thing you want them to believe about you.

The other weakness answer, if you need a second**Say it like this**

“Breadth versus depth. I’ve built across data engineering, ML, C++ and interface design, and the cost of that is that I’m not the deepest person in the room on any one of them. On a small team that’s an asset; on a team that needs a specialist it’s a real gap, and I’d rather say so. What I’d want from a first role is somewhere to go deep on one of them.”

14. The general question bank

14.1 Motivation and fit

Q. Why did you apply here?

A. Never generic. Three ingredients: something specific about *their* problem, why it connects to something you have actually done, and what you want from the role. Research the company enough to name one real thing.

Q. Why should we hire a graduate rather than someone with experience?

A. “For the same money you get someone who has already shipped four systems end to end, has worked in industry for two years before the masters, and has no habits to unlearn from a stack that isn’t yours. What I don’t have is scale — I’ve never operated something with a thousand users — and I’d expect to learn that from you rather than pretend otherwise.”

Q. Where do you see yourself in five years?

A. Avoid both extremes — “running the company” and “I don’t know”. Something like: “owning a system end to end rather than a component of one, and being the person a team trusts on data correctness. Whether that’s a senior engineer or a lead depends on what the team needs.”

Q. What are you looking for in a first role?

A. Ownership, feedback, and shipping to production early. Say it plainly: “I’d rather build one thing that’s used than rotate through four that aren’t.”

Q. What’s your favourite thing you’ve built and why?

A. Pick by role (§3) and give a *reason* that reveals your values — “GridPulse, because it’s the one where I had to decide what correct meant before I could build anything.”

14.2 Technical judgement

Q. How do you decide whether a problem needs machine learning?

A. “Three tests. Is there a pattern too complex to write down as rules? Do I have labelled examples of the right answer? And can I tolerate being wrong sometimes — because a model will be. If any of those is no, rules are better: they’re inspectable, testable and they don’t drift. GridPulse has no model in it for exactly that reason.”

Q. How do you approach a problem you’ve never seen before?

A. “Reproduce or characterise it first — I want the smallest concrete instance I can look at. Then find out whether it’s a solved problem, because most are and the literature is cheaper than my time. Then build the smallest version that could possibly work, and only add complexity where the simple version measurably fails.”

Q. What does “production” mean to you?

A. “Four things, and a demo has none of them. It runs without me. It fails loudly rather than quietly. Someone other than me can rebuild it from the repository. And there’s a test that tells me when I’ve broken it. My projects have all four, which is the difference between them and a notebook.”

Q. How do you keep up with the field?

A. Be specific and modest. Name what you actually read or follow, and say that you learn by building rather than by reading — then point at the projects as evidence.

Q. What's a technical opinion you hold that others disagree with?

A. Have one ready. A good, defensible option: *“that most teams should write far more data tests and far fewer unit tests than they do. Unit tests check the code you wrote; data tests check the assumptions you didn't know you were making, and that's where the expensive failures are.”*

14.3 Curveballs

Q. What would you do in your first 30 days?

A. “Read the code and the data model before proposing anything. Find out what breaks most often and what people are quietly working around. Ship something small and real in the first fortnight, even if it's minor — I'd rather establish that I can land a change than spend a month writing a strategy nobody asked for.”

Q. Something on your CV I should ask you about that I haven't?

A. A gift. Have one prepared: the teaching, if the role touches communication or product; the GridPulse validation layer, if it's a data role; the SignSpeak pivot, if it's an ML role.

Q. Do you have any concerns about this role?

A. Answer honestly with something small and real, and pair it with what would resolve it: “the thing I'd want to understand is how much mentoring is realistically available, because I'll be learning quickly and I don't want to be guessing.” A flat “no” wastes the question.

Q. Why are there gaps / why did you leave that role after six months?

A. Flat, factual, no apology. “I'd been accepted into the masters at Sydney and moved to Australia.” Then stop talking.

15. Questions you ask them

Not asking questions reads as not being interested. Asking generic ones reads as not having prepared. Two rules: ask what you actually want to know, and ask things that only *they* can answer.

15.1 The strongest three

1. **“What does the first thing I ship look like, and roughly when?”**
Tests whether they have thought about onboarding, and signals that you expect to ship.
2. **“What’s the part of the system you’d most like someone to fix but nobody’s had time for?”**
You will learn more from the answer to this than from anything on the job ad — it tells you the real state of the codebase, and it’s a question only an engineer asks.
3. **“How do you know when something you’ve shipped is wrong?”**
Directly on your theme, and their answer tells you whether their engineering culture matches yours.

15.2 The reserve list

- “How is the team structured, and who would I be learning from?”
- “What does a code review look like here?”
- “What’s changed most about the product in the last six months?”
- “What separates someone who does well in this role from someone who struggles?”
- “How much of the work is new build versus maintaining what exists?”
- “What’s the deployment process — how often do changes go out?”

15.3 Save these for later stages

Salary band, leave, remote/hybrid split, promotion timelines, equity. Asking at a first screen is not fatal but it spends goodwill you’d rather keep. The exception: if a recruiter asks *you* for a salary expectation, answer with a researched range and add “and I’m flexible for the right role” — do not say “whatever you think is fair”.

16. The consistency audit — do this before any interview

An inconsistency between your CV and your own account of your own work is uniquely damaging, because it is the one error an interviewer cannot attribute to inexperience. Fix these before you need them.

Item	Action
SignSpeak accuracy	CV says 99.2% ; the repository README reports 0.9943 test accuracy / 0.9941 macro F1. Decide which is current and make every document agree.
Airbnb accuracy	CV says “80%”; repository says 0.798 . Consistent, but always quote it with the 0.390 baseline .
GridPulse authorship	The repository lists two authors. Have a precise, honest sentence about which components were yours.
The leakage cause	Verify against your own commit history that the failure was preprocessing-before-split. If the real cause differed, <i>tell the real one</i> — the story is just as good and the risk vanishes.
The 40% and 30% figures	Magic Software throughput and HPE resolution time. Know how each was measured and whether it was your measurement or a team dashboard.
The IEEE paper	Re-read it. Know the claim, your contribution, and one limitation.
Live app health	Open all three Streamlit URLs the day before. Free-tier apps sleep and occasionally break on dependency updates. An interviewer clicking a dead link is an avoidable own goal.
README currency	An interviewer who looks at your GitHub will read the README, not the code. Make sure each one leads with what the project is and what the honest numbers are.

The rule

Every number you say out loud must match a number someone can find. If you cannot verify a figure, either verify it or stop quoting it. There is no third option that survives a follow-up question.

16.1 A four-day preparation plan

Day	What to do
Day 1	Run the consistency audit above, end to end. Fix every discrepancy in the source documents. Open all three apps. Re-read the IEEE paper.
Day 2	Projects. For each of the four, say the 60-second version out loud without notes, then draw the architecture diagram from memory on paper. Repeat until both are fluent.
Day 3	Question banks. Answer every question in §4–§7 out loud <i>before</i> reading the answer. Note the ones you fumbled and only re-drill those.
Day 4	Concepts (§12) and behavioural (§13). Then research the specific company, and write the three questions you’ll ask them.
Morning of	Section 17 only. Nothing new.

The most valuable single exercise

Draw each architecture diagram from memory, on paper, out loud. Not read — drawn. If you can draw GridPulse’s six stages and say why each one is separate, you can survive any depth of questioning about it, because you are reasoning from the structure rather than recalling a script. Candidates who have memorised answers fall apart on the second follow-up. Candidates who understand the shape do not.

17. The one page — print this

Cheat sheet

Read this and nothing else on the morning.

The through-line — open with this

“I build AI systems end to end, and I care most about the part that makes them trustworthy: knowing whether the number is actually right.”

The numbers

GridPulse	668,134 readings · 541 facilities · 5 regions · 5-min grain · 7-day window 3.62 TWh · 2.15 Mt CO ₂ e · coal ≈58% energy / ≈95% emissions 9 quality gates + 39 dbt tests, all passing · DuckDB, dbt, Dagster
Airbnb	17,701 listings · 24 features · 3 tiers (31/39/30%) · 3,541 test rows 0.798 accuracy vs 0.390 baseline · 0.802 macro F1 · 0.930 macro ROC-AUC LR 0.767 → RF 0.792 → XGBoost 0.798 · non-adjacent errors: 0.5%
SignSpeak	24 letters · 21 landmarks · 76 features · 4,412 samples SVM RBF $C=30$ · 0.9943 accuracy · 13.4 ms/frame · 2.3 MB
Crypto	AES + RSA hybrid · 1 GB under a minute · ~3× faster · C++/OpenSSL
Experience	HPE 20+ customers, 90%+ SLA, ~30% faster · Magic 5+ features, ~40%, 30+ defects Teaching 50+ learners, 15+ projects · VIIT CGPA 9.4/10 · IEEE ICBDS 2023, cited 7

Say it like this

Tell me about yourself (30s). “I’m finishing a Master of Computer Science in Data Science and AI at Sydney in December 2026. Before that I was a C++ developer at Magic Software and a technical consultant at HPE. What I actually do is build AI and data systems end to end and put them in front of people — three live apps: a data platform over the Australian electricity market, an ML pricing tool for Airbnb hosts, and a real-time sign-language-to-speech app. The thread is that I care whether the number is right, not just whether the demo works.”

Say it like this

Your failure story (30s). “I shipped an XGBoost pricing model and then found it had been cheating — I’d fitted my scaling and encoding before splitting train from test, so my held-out set was never held out. Rebuilding preprocessing inside each CV fold dropped the score to 79.8% against a 39% baseline: a worse number and a real one. Then I added tests in CI that fail if a transformer is ever fitted outside a fold, so the mistake is unrepeatable rather than merely fixed.”

Say it like this

Your engineering story (30s). “A scheduled load in my energy pipeline failed partway through and didn’t error — it just wrote fewer rows, and the dashboard published a completely plausible wrong number. A crash is obvious; a quietly wrong number gets believed. So I built the data-quality tests as a gate: a partial load now fails loudly and publishes nothing.”

The three phrases to land	The three things not to do
“A crash is obvious; a quietly wrong number gets believed.”	Don’t say a number without its baseline.
“A worse number, and a real one.”	Don’t say “we” when you mean “I”.
“The smaller model wasn’t a compromise — the features were better.”	Don’t bluff a term. “I don’t know that one — what does it mean in your setup?” is a good answer.

Availability: part-time from September 2026, full-time from December 2026. Temporary Graduate visa (subclass 485), full work rights. Based in Sydney.

Links: github.com/adhu-601 · [GridPulse](#) · [Airbnb](#) · [SignSpeak](#) · [IEEE paper](#) · [LinkedIn](#)

Ask them: What does the first thing I ship look like? · What would you most like someone to fix that nobody’s had time for? · How do you know when something you’ve shipped is wrong?